

SOLIDWORKS API (.NET VBA C#) — Examples (2/50): Add .NET Controls to SolidWorks Using an Add_and_Remove_Items_to_File_Save_As_and

SOLIDWORKS API Documentation (converted from CHM)

Add .NET Controls to SOLIDWORKS Design using an Add-in Example (VB.NET)

This example shows how to add .NET controls to FeatureManager, PropertyManager, Task Pane, and model view using an add-in.

```

'-----
' Preconditions:
' 1. In Microsoft Visual Studio 2012 or later, create a new project using
'    the SwVBAddin template.
' 2. Name the project VBDotNetControlsDemo.
' 3. Select Debug in the Solution Configurations drop-down list.
' 4. Click Project > VBDotNetControlsDemo Properties.
'    a. On the Application tab, verify that Target framework is
'       .NET Framework 3.0 or later.
'    b. On the Debug tab, select Start external program and type the
'       pathname of your SOLIDWORKS Design executable.
' 5. Save the project.
' 6. In the Solution Explorer, add the following .NET Framework references:
'    * PresentationCore
'    * PresentationFramework
'    * UIAutomationProvider
'    * WindowsBase
'    * WindowsFormsIntegration
'    * System.Xaml
' 7. Copy and paste:
'    a. this into SwAddin.vb.
'    b. this into UserPMPPage.vb.
'    c. this into PMPHandler.vb.
' 8. Modify install_dir in SwAddin::WinFormInFeatureMgr to point to your
'    SOLIDWORKS Design install directory.
' 9. Add a Windows Presentation Foundation User Control to the project:
'    a. In the Solution Explorer, right-click the project and click
'       Add > User Control.
'    b. Type WPFFControl.xaml in Name.
'    c. Click the User Control (WPF) template.
'    d. Click Add.
'    e. Copy and paste this into WPFFControl.xaml.
'    f. Expand WPFFControl.xaml in the Solution Explorer and copy
'       and paste this into WPFFControl.xaml.vb.
' 10. Add a Windows Forms User Control to the project:
'    a. In the Solution Explorer, right-click the project and click
'       Add > User Control.
'    b. Click Add.
'    c. Copy and paste this into UserControl1.vb.
'    d. Expand UserControl1.vb in the Solution Explorer and copy
'       and paste this into UserControl1.Designer.vb.
' 11. Create a Windows Form:
'    a. In the Solution Explorer, right-click the project and select
'       Add > Windows Form.
'    b. Click Add.
'    c. Copy and paste this into Form1.vb.
'    d. Expand Form1.vb in the Solution Explorer and copy
'       and paste this into Form1.Designer.vb.
' 12. Save, clean, and build the project.
'-----
' Postconditions:
' 1. In SOLIDWORKS Design, click Tools > VBDotNetControlsDemo > WinFormInTaskPane.
' 2. Displays a new tab with .NET controls in the Task Pane.
' 3. Open a model document.
' 4. Click Tools > VBDotNetControlsDemo > UserControlInModelView.
' 5. Displays a .NET User Control1 tab below the model view.
' 6. Click Tools > VBDotNetControlsDemo > WPFFInModelView.
' 7. Displays a .NET WPF Control tab below the model view.
' 8. Click Tools > VBDotNetControlsDemo > WinFormInFeatureMgr.
' 9. Displays a new view in a split panel of the FeatureManager design tree.
' 10. Click Tools > VBDotNetControlsDemo > Show PMP.
' 11. Click OK in each of the three message boxes.
' 12. Displays a new tab with .NET controls in the PropertyManager.
'-----

'SwAddin.vb
Imports System
Imports System.Collections
Imports System.Reflection
Imports System.Runtime.InteropServices

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports SolidWorksTools
Imports SolidWorksTools.File
Imports System.Windows.Forms
Imports System.Windows.Forms.Integration

Imports System.Collections.Generic
Imports System.Diagnostics

<Guid("1bce57ff-da20-460b-9c88-32f62e819510")> _
<ComVisible(True)> _
<SwAddin( _

```

```

        Description:="VBDotNetControlsDemo description", _
        Title:="VBDotNetControlsDemo ", _
        LoadAtStartup:=True _
    )> _
    Public Class SwAddin
Implements SolidWorks.Interop.swpublished.SwAddin

#Region "Local Variables"
Dim WithEvents iSwApp As SldWorks
Dim iCmdMgr As ICommandManager
Dim cmdGroup As ICommandGroup
Dim addinID As Integer
Dim openDocs As Hashtable
Dim SwEventPtr As SldWorks
Dim ppage As UserPMPPage
Dim iBmp As BitmapHandler
Dim TaskPanWinFormControl As Form1
Dim TaskPanUserControl As UserControl1
Dim FeatureMgrControl As Form1
Dim ModelView1Control As UserControl1
Dim WpfModelView1Control As WPFControl
Dim ModelView1host As ElementHost

#Region "Event Handler Variables"
Dim m_openDocs As Hashtable
#End Region

' Public Properties
ReadOnly Property SwApp() As SldWorks
    Get
        Return iSwApp
    End Get
End Property

ReadOnly Property CmdMgr() As ICommandManager
    Get
        Return iCmdMgr
    End Get
End Property

ReadOnly Property OpenDocumentsTable() As Hashtable
    Get
        Return openDocs
    End Get
End Property
#End Region

#Region "SolidWorks Registration"

<ComRegisterFunction()> Public Shared Sub RegisterFunction(ByVal t As Type)

    ' Get Custom Attribute: SwAddinAttribute
    Dim attributes() As Object
    Dim SwAttr As SwAddinAttribute = Nothing

    attributes = System.Attribute.GetCustomAttributes(GetType(SwAddin), GetType(SwAddinAttribute))

    If attributes.Length > 0 Then
        SwAttr = DirectCast(attributes(0), SwAddinAttribute)
    End If
    Try
        Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SolidWorks\Addins\" + t.Guid.ToString() + "}"
        Dim addinkey As Microsoft.Win32.RegistryKey = hklm.CreateSubKey(keyname)
        addinkey.SetValue(Nothing, 0)
        addinkey.SetValue("Description", SwAttr.Description)
        addinkey.SetValue("Title", SwAttr.Title)

        keyname = "Software\SolidWorks\AddInsStartup\" + t.Guid.ToString() + "}"
        addinkey = hkcu.CreateSubKey(keyname)
        addinkey.SetValue(Nothing, SwAttr.LoadAtStartup, Microsoft.Win32.RegistryValueKind.DWord)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem registering this dll: SwAttr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: SwAttr is null.\n " & nl.Message)
    Catch e As System.Exception
        Console.WriteLine("There was a problem registering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem registering this dll: " & e.Message)
    End Try
End Sub

<ComUnregisterFunction()> Public Shared Sub UnregisterFunction(ByVal t As Type)
    Try
        Dim hklm As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.LocalMachine
        Dim hkcu As Microsoft.Win32.RegistryKey = Microsoft.Win32.Registry.CurrentUser

        Dim keyname As String = "SOFTWARE\SolidWorks\Addins\" + t.Guid.ToString() + "}"
        hklm.DeleteSubKey(keyname)

        keyname = "Software\SolidWorks\AddInsStartup\" + t.Guid.ToString() + "}"
        hkcu.DeleteSubKey(keyname)
    Catch nl As System.NullReferenceException
        Console.WriteLine("There was a problem unregistering this dll: SwAttr is null.\n " & nl.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: SwAttr is null.\n " & nl.Message)
    Catch e As System.Exception

```

```

        Console.WriteLine("There was a problem unregistering this dll: " & e.Message)
        System.Windows.Forms.MessageBox.Show("There was a problem unregistering this dll: " & e.Message)
    End Try

End Sub

#End Region

#Region "ISwAddin Implementation"

Function ConnectToSW(ByVal ThisSW As Object, ByVal Cookie As Integer) As Boolean Implements SolidWorks.Interop.swpublished.SwAddin
    iSwApp = ThisSW
    addinID = Cookie

    ' Set up callbacks
    iSwApp.SetAddinCallbackInfo(0, Me, addinID)

    ' Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(Cookie)
    AddCommandMgr()

    'Setup the Event Handlers
    SwEventPtr = iSwApp
    openDocs = New Hashtable
    AttachEventHandlers()

    'Setup Sample Property Manager
    AddPMP()

    ConnectToSW = True
End Function

Function DisconnectFromSW() As Boolean Implements SolidWorks.Interop.swpublished.SwAddin.DisconnectFromSW
    RemoveCommandMgr()
    RemovePMP()
    DetachEventHandlers()

    System.Runtime.InteropServices.Marshal.ReleaseComObject(iCmdMgr)
    iCmdMgr = Nothing
    System.Runtime.InteropServices.Marshal.ReleaseComObject(iSwApp)
    iSwApp = Nothing
    'The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
    GC.Collect()
    GC.WaitForPendingFinalizers()

    GC.Collect()
    GC.WaitForPendingFinalizers()

    DisconnectFromSW = True
End Function
#End Region

#Region "UI Methods"
Public Sub AddCommandMgr()

    Dim iBmp As New BitmapHandler()

    Dim thisAssembly As Assembly
    Dim cmdIndex0 As Integer, cmdIndex1 As Integer, cmdIndex2 As Integer, cmdIndex3 As Integer, cmdIndex4 As Integer
    Dim Title As String = "VBDotNetControlsDemo ", ToolTip As String = "VBDotNetControlsDemo "

    Dim docTypes As Integer() = New Integer() {swDocumentTypes_e.swDocASSEMBLY, swDocumentTypes_e.swDocDRAWING}, swDocumentTypes

    thisAssembly = System.Reflection.Assembly.GetAssembly(Me.[GetType]())

    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1)
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("VBDotNetControlsDemo .ToolbarLarge.bmp", thisAssembly)
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("VBDotNetControlsDemo .ToolbarSmall.bmp", thisAssembly)
    cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("VBDotNetControlsDemo .MainIconLarge.bmp", thisAssembly)
    cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("VBDotNetControlsDemo .MainIconSmall.bmp", thisAssembly)

    cmdIndex0 = cmdGroup.AddCommandItem("WinFromInTaskPane", -1, "Add Windows Form In Task Pane", "WinFormInTaskPane", 0, "WinFormInTaskPane", 0)
    cmdIndex1 = cmdGroup.AddCommandItem("UserControlInModelView", -1, "Add User Control In Model View", "UserControlInModelView", 1, "UserControlInModelView", 1)
    cmdIndex2 = cmdGroup.AddCommandItem("WPFInModelView", -1, "Add WPF In ModelView", "WPFInModelView", 2, "WPFInModelView", 2)
    cmdIndex3 = cmdGroup.AddCommandItem("WinFormInFeatureMgr", -1, "Add Windows Form In FeatureManager design tree", "WinFormInFeatureMgr", 3, "WinFormInFeatureMgr", 3)
    cmdIndex4 = cmdGroup.AddCommandItem("Show PMP", -1, "Display PropertyManager page with .NET Controls", "Show PMP", 4, "Show PMP", 4)

    cmdGroup.HasToolbar = True
    cmdGroup.HasMenu = True
    cmdGroup.Activate()

    Dim bResult As Boolean

    For Each type As Integer In docTypes
        Dim cmdTab As ICommandTab

        cmdTab = iCmdMgr.GetCommandTab(type, Title)
    End For
End Sub

```

```

If cmdTab Is Nothing Then
    cmdTab = DirectCast(iCmdMgr.AddCommandTab(type, Title), ICommandTab)

    Dim cmdBox As CommandTabBox = cmdTab.AddCommandTabBox()

    Dim cmdIDs As Integer() = New Integer(5) {}
    Dim TextType As Integer() = New Integer(5) {}

    cmdIDs(0) = cmdGroup.CommandID(cmdIndex0)
    TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

    cmdIDs(1) = cmdGroup.CommandID(cmdIndex1)
    TextType(1) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

    cmdIDs(2) = cmdGroup.CommandID(cmdIndex2)
    TextType(2) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

    cmdIDs(3) = cmdGroup.CommandID(cmdIndex3)
    TextType(3) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

    cmdIDs(4) = cmdGroup.CommandID(cmdIndex4)
    TextType(4) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal

    cmdIDs(5) = cmdGroup.ToolbarId
    TextType(5) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal Or swCommandTabButtonFlyoutStyle_e.s

    bResult = cmdBox.AddCommands(cmdIDs, TextType)

    Dim cmdBox1 As CommandTabBox = cmdTab.AddCommandTabBox()
    cmdIDs = New Integer(0) {}
    TextType = New Integer(0) {}

    cmdIDs(0) = cmdGroup.ToolbarId
    TextType(0) = swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow Or swCommandTabButtonFlyoutStyle_e.swComm

    bResult = cmdBox1.AddCommands(cmdIDs, TextType)

    cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId)

End If
Next
thisAssembly = Nothing
iBmp.Dispose()
End Sub

Public Sub RemoveCommandMgr()
    iCmdMgr.RemoveCommandGroup(1)
End Sub

Public Function AddPMP() As [Boolean]
    ppage = New UserPMPPage(Me)
    Return True
End Function

Public Function RemovePMP() As [Boolean]
    ppage = Nothing
    Return True
End Function

#End Region

#Region "UI Callbacks"
Public Sub WinFormInTaskPane()
    Dim pTaskPanView As ITaskpaneView
    pTaskPanView = iSwApp.CreateTaskpaneView2("", ".NET Windows Form Control")
    TaskPanWinFormControl = New Form1()
    pTaskPanView.DisplayWindowFromHandle64(TaskPanWinFormControl.Handle.ToInt64())
End Sub

Public Function EnableWinFormInTaskPane() As Integer
    Return 1
End Function

Public Sub UserControlInModelView()
    Dim pDoc As IModelDoc2
    pDoc = DirectCast(iSwApp.ActiveDoc, IModelDoc2)
    Dim swModelViewMgr As IModelViewManager
    swModelViewMgr = pDoc.ModelViewManager
    ModelView1Control = New UserControl1()
    swModelViewMgr.DisplayWindowFromHandle64(".NET User Control1", ModelView1Control.Handle.ToInt64(), False)
End Sub

Public Function EnableUserControlInModelView() As Integer
    If iSwApp.ActiveDoc IsNot Nothing Then
        Return 1
    Else
        Return 0
    End If
End Function

```

```

Public Sub WPFinModelView()
    Dim pDoc As IModelDoc2
    pDoc = DirectCast(iSwApp.ActiveDoc, IModelDoc2)
    Dim swModelViewMgr As IModelViewManager
    swModelViewMgr = pDoc.ModelViewManager
    WpfModelView1Control = New WPFControl()

    ModelView1Host = New ElementHost()
    ModelView1Host.Child = WpfModelView1Control
    swModelViewMgr.DisplayWindowFromHandle64(".NET WPF Control", ModelView1Host.Handle.ToInt64(), True)
End Sub

Public Function EnableWPFinModelView() As Integer
    If iSwApp.ActiveDoc IsNot Nothing Then
        Return 1
    Else
        Return 0
    End If
End Function

Public Sub WinFormInFeatureMgr()
    Dim pDoc As IModelDoc2
    pDoc = DirectCast(iSwApp.ActiveDoc, IModelDoc2)
    Dim swModelViewMgr As IModelViewManager
    swModelViewMgr = pDoc.ModelViewManager
    Dim swFeatMgrTabTop As IFeatMgrView
    FeatureMgrControl = New Form1()
    swFeatMgrTabTop = swModelViewMgr.CreateFeatureMgrWindowFromHandle64("public_documents\samples\tutorial\api\addin\pm_extrude
FeatureMgrControl.Handle.ToInt64(), ".NET Windows Form Control", swFeatMgrPane_e.swFeatMgrPaneTop)
    pDoc.FeatureManagerSplitterPosition = 0.5
    swFeatMgrTabTop.ActivateView()
End Sub

Public Function EnableWinFormInFeatureMgr() As Integer
    If iSwApp.ActiveDoc IsNot Nothing Then
        Return 1
    Else
        Return 0
    End If
End Function

Public Sub ShowPMP()
    If ppage IsNot Nothing Then
        ppage.Show()
    End If
End Sub

Public Function EnablePMP() As Integer
    If iSwApp.ActiveDoc IsNot Nothing Then
        Return 1
    Else
        Return 0
    End If
End Function
#End Region

#Region "Event Methods"
Sub AttachEventHandlers()
    AttachSWEvents()

    'Listen for events on all currently open docs
    AttachEventsToAllDocuments()
End Sub

Sub DetachEventHandlers()
    DetachSWEvents()

    'Close events on all currently open docs
    Dim docHandler As DocumentEventHandler
    Dim key As ModelDoc2
    Dim numKeys As Integer
    numKeys = openDocs.Count
    If numKeys > 0 Then
        Dim keys() As Object = New Object(numKeys - 1) {}

        'Remove all document event handlers
        openDocs.Keys.CopyTo(keys, 0)
        For Each key In keys
            docHandler = openDocs.Item(key)
            docHandler.DetachEventHandlers() 'This also removes the pair from the hash
            docHandler = Nothing
            key = Nothing
        Next
    End If
End Sub

Sub AttachSWEvents()
    Try
        AddHandler iSwApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        AddHandler iSwApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        AddHandler iSwApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        AddHandler iSwApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        AddHandler iSwApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

```

```

    End Try
End Sub

Sub DetachSWEvents()
    Try
        RemoveHandler iSWApp.ActiveDocChangeNotify, AddressOf Me.SldWorks_ActiveDocChangeNotify
        RemoveHandler iSWApp.DocumentLoadNotify2, AddressOf Me.SldWorks_DocumentLoadNotify2
        RemoveHandler iSWApp.FileNewNotify2, AddressOf Me.SldWorks_FileNewNotify2
        RemoveHandler iSWApp.ActiveModelDocChangeNotify, AddressOf Me.SldWorks_ActiveModelDocChangeNotify
        RemoveHandler iSWApp.FileOpenPostNotify, AddressOf Me.SldWorks_FileOpenPostNotify
    Catch e As Exception
        Console.WriteLine(e.Message)
    End Try
End Sub

Sub AttachEventsToAllDocuments()
    Dim modDoc As ModelDoc2
    modDoc = iSWApp.GetFirstDocument()
    While Not modDoc Is Nothing
        If Not openDocs.Contains(modDoc) Then
            AttachModelDocEventHandler(modDoc)
        Else
            Dim docHandler As DocumentEventHandler = openDocs(modDoc)
            If Not docHandler Is Nothing Then
                docHandler.ConnectModelViews()
            End If
        End If
        modDoc = modDoc.GetNext()
    End While
End Sub

Function AttachModelDocEventHandler(ByVal modDoc As ModelDoc2) As Boolean
    If modDoc Is Nothing Then
        Return False
    End If
    Dim docHandler As DocumentEventHandler = Nothing

    If Not openDocs.Contains(modDoc) Then
        Select Case modDoc.GetType
            Case swDocumentTypes_e.swDocPART
                docHandler = New PartEventHandler()
            Case swDocumentTypes_e.swDocASSEMBLY
                docHandler = New AssemblyEventHandler()
            Case swDocumentTypes_e.swDocDRAWING
                docHandler = New DrawingEventHandler()
        End Select

        docHandler.Init(iSWApp, Me, modDoc)
        docHandler.AttachEventHandlers()
        openDocs.Add(modDoc, docHandler)
    End If
End Function

Sub DetachModelEventHandler(ByVal modDoc As ModelDoc2)
    Dim docHandler As DocumentEventHandler
    docHandler = openDocs.Item(modDoc)
    openDocs.Remove(modDoc)
    modDoc = Nothing
    docHandler = Nothing
End Sub
#End Region

#Region "Event Handlers"
Function SldWorks_ActiveDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

Function SldWorks_DocumentLoadNotify2(ByVal docTitle As String, ByVal docPath As String) As Integer
End Function

Function SldWorks_FileNewNotify2(ByVal newDoc As Object, ByVal doctype As Integer, ByVal templateName As String) As Integer
    AttachEventsToAllDocuments()
End Function

Function SldWorks_ActiveModelDocChangeNotify() As Integer
    'TODO: Add your implementation here
End Function

Function SldWorks_FileOpenPostNotify(ByVal FileName As String) As Integer
    AttachEventsToAllDocuments()
End Function
#End Region
End Class

```

[Back to top](#)

'UserPMPage.vb

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swpublished
Imports SolidWorks.Interop.swconst
Imports System.Windows.Forms

```

```

Imports System.Windows.Forms.Integration

Public Class UserPMPPage
    'Local Objects
    Dim swPropertyPage As IPropertyManagerPage2
    Dim handler As PMPHandler
    Dim iSwApp As ISldWorks
    Dim userAddin As SwAddin

    Dim MyWPFFControl As WPFControl
    Dim elhost As ElementHost
    Dim MyUserControl As UserControl1
    Dim MyWinFormControl As Form1

    #Region "Property Manager Page Controls"
    'Groups
    Dim group1 As IPropertyManagerPageGroup
    Dim group2 As IPropertyManagerPageGroup
    Dim group3 As IPropertyManagerPageGroup

    'Controls
    Dim dotnet1 As IPropertyManagerPageWindowFromHandle
    Dim dotnet2 As IPropertyManagerPageWindowFromHandle
    Dim dotnet3 As IPropertyManagerPageWindowFromHandle

    'Control IDs
    Public Const group1ID As Integer = 0
    Public Const group2ID As Integer = 1
    Public Const group3ID As Integer = 2

    Public Const DotNet1ID As Integer = 3
    Public Const DotNet2ID As Integer = 4
    Public Const DotNet3ID As Integer = 5

    #End Region

    Public Sub New(addin As SwAddin)
        userAddin = addin
        iSwApp = DirectCast(userAddin.SwApp, ISldWorks)
        CreatePropertyManagerPage()
    End Sub

    Protected Sub CreatePropertyManagerPage()
        Dim errors As Integer = -1
        Dim options As Integer = swPropertyManagerPageOptions_e.swPropertyManagerOptions_OkayButton Or swPropertyManagerPageOptions_

        handler = New PMPHandler(userAddin)
        swPropertyPage = DirectCast(iSwApp.CreatePropertyManagerPage(".Net PMP control", options, handler, errors), IPropertyManager
        If swPropertyPage IsNot Nothing AndAlso errors = swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay Then
            Try
                AddControls()
            Catch e As Exception
                iSwApp.SendMsgToUser2(e.Message, 0, 0)
            End Try
        End If
    End Sub

    'Controls display on the page top to bottom in the order
    'in which they are added to the page
    Protected Sub AddControls()
        Dim controlType As Short = -1
        Dim align As Short = -1
        Dim options As Integer = -1

        'Add the groups
        options = swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded Or swAddGroupBoxOptions_e.swGroupBoxOptions_Visible

        group1 = DirectCast(swPropertyPage.AddGroupBox(group1ID, "Windows Form controls", options), IPropertyManagerPageGroup)

        options = swAddGroupBoxOptions_e.swGroupBoxOptions_Checkbox Or swAddGroupBoxOptions_e.swGroupBoxOptions_Visible

        group2 = DirectCast(swPropertyPage.AddGroupBox(group2ID, "User controls", options), IPropertyManagerPageGroup)

        options = swAddGroupBoxOptions_e.swGroupBoxOptions_Checkbox Or swAddGroupBoxOptions_e.swGroupBoxOptions_Visible

        group3 = DirectCast(swPropertyPage.AddGroupBox(group3ID, "WPF controls", options), IPropertyManagerPageGroup)

        controlType = swPropertyManagerPageControlType_e.swControlType_WindowFromHandle
        align = swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge
        options = swAddControlOptions_e.swControlOptions_Enabled Or swAddControlOptions_e.swControlOptions_Visible

        dotnet1 = DirectCast(group1.AddControl2(DotNet1ID, controlType, ".Net Windows Form control", align, options, "This control i
        dotnet1.Height = 150

        controlType = swPropertyManagerPageControlType_e.swControlType_WindowFromHandle
        align = swPropertyManagerPageControlLeftAlign_e.swControlAlign_LeftEdge
        options = swAddControlOptions_e.swControlOptions_Enabled Or swAddControlOptions_e.swControlOptions_Visible

        dotnet2 = DirectCast(group2.AddControl2(DotNet2ID, controlType, ".Net user form control", align, options, "This control is a
        dotnet2.Height = 150

        dotnet3 = DirectCast(group3.AddControl2(DotNet3ID, controlType, ".Net WPF control", align, options, "This control is added v
        dotnet3.Height = 50
    
```

```

End Sub

Public Sub Show()

    'Windows Form control
    MyWinFormControl = New Form1()
    'If you are adding Windows form in the PropertyManager Page, set TopLevel to false
    MyWinFormControl.TopLevel = False
    MyWinFormControl.Show()
    dotnet1.SetWindowHandle64(MyWinFormControl.Handle.ToInt64())

    'User control
    MyUserControl = New UserControl1()
    dotnet2.SetWindowHandle64(MyUserControl.Handle.ToInt64())

    'WPF control
    elhost = New ElementHost()
    MyWPFCtrl = New WPFCtrl()
    elhost.Child = MyWPFCtrl
    dotnet3.SetWindowHandle64(elhost.Handle.ToInt64())

    'Show PropertyManager page
    swPropertyPage.Show()

End Sub
End Class

```

[Back to top](#)

```

'PMPHandler.vb
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System.Windows.Forms

Public Class PMPHandler
    Implements PropertyManagerPage2Handler9

    Dim iSwApp As SldWorks
    Dim userAddin As SwAddin

    Public Sub New(addin As SwAddin)
        userAddin = addin
        iSwApp = DirectCast(userAddin.SwApp, ISldWorks)
    End Sub

    'Implement these methods from the interface
    Sub AfterClose() Implements PropertyManagerPage2Handler9.AfterClose
        ''This function must contain code, even if it does nothing, to prevent the
        '' .NET runtime environment from doing garbage collection at the wrong time.
        Dim IndentSize As Integer
        IndentSize = System.Diagnostics.Debug.IndentSize
        System.Diagnostics.Debug.WriteLine(IndentSize)

    End Sub

    Sub OnCheckboxCheck(ByVal id As Integer, ByVal status As Boolean) Implements PropertyManagerPage2Handler9.OnCheckboxCheck

    End Sub

    Sub OnClose(ByVal reason As Integer) Implements PropertyManagerPage2Handler9.OnClose
        ''This function must contain code, even if it does nothing, to prevent the
        '' .NET runtime environment from doing garbage collection at the wrong time.
        Dim IndentSize As Integer
        IndentSize = System.Diagnostics.Debug.IndentSize
        System.Diagnostics.Debug.WriteLine(IndentSize)
    End Sub

    Sub OnComboboxEditChanged(ByVal id As Integer, ByVal text As String) Implements PropertyManagerPage2Handler9.OnComboboxEditChanged

    End Sub

    Function OnActiveXControlCreated(ByVal id As Integer, ByVal status As Boolean) As Integer Implements PropertyManagerPage2Handler9.OnActiveXControlCreated
        OnActiveXControlCreated = -1
    End Function

    Sub OnButtonPress(ByVal id As Integer) Implements PropertyManagerPage2Handler9.OnButtonPress

    End Sub

    Sub OnComboboxSelectionChanged(ByVal id As Integer, ByVal item As Integer) Implements PropertyManagerPage2Handler9.OnComboboxSelectionChanged

    End Sub

    Sub OnGroupCheck(ByVal id As Integer, ByVal status As Boolean) Implements PropertyManagerPage2Handler9.OnGroupCheck

```

```

End Sub

Sub OnGroupExpand(ByVal id As Integer, ByVal status As Boolean) Implements PropertyManagerPage2Handler9.OnGroupExpand

End Sub

Function OnHelp() As Boolean Implements PropertyManagerPage2Handler9.OnHelp
    OnHelp = True
End Function

Sub OnListboxSelectionChanged(ByVal id As Integer, ByVal item As Integer) Implements PropertyManagerPage2Handler9.OnListboxSele

End Sub

Function OnNextPage() As Boolean Implements PropertyManagerPage2Handler9.OnNextPage
    OnNextPage = True
End Function

Sub OnNumberboxChanged(ByVal id As Integer, ByVal val As Double) Implements PropertyManagerPage2Handler9.OnNumberboxChanged

End Sub

Sub OnOptionCheck(ByVal id As Integer) Implements PropertyManagerPage2Handler9.OnOptionCheck

End Sub

Function OnPreviousPage() As Boolean Implements PropertyManagerPage2Handler9.OnPreviousPage
    OnPreviousPage = True
End Function

Sub OnSelectionboxCalloutCreated(ByVal id As Integer) Implements PropertyManagerPage2Handler9.OnSelectionboxCalloutCreated

End Sub

Sub OnSelectionboxCalloutDestroyed(ByVal id As Integer) Implements PropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed

End Sub

Sub OnSelectionboxFocusChanged(ByVal Id As Integer) Implements PropertyManagerPage2Handler9.OnSelectionboxFocusChanged

End Sub

Sub OnSelectionboxListChanged(ByVal id As Integer, ByVal item As Integer) Implements PropertyManagerPage2Handler9.OnSelectionbox

End Sub

Sub OnTextboxChanged(ByVal id As Integer, ByVal text As String) Implements PropertyManagerPage2Handler9.OnTextboxChanged

End Sub

Public Sub AfterActivation() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterActivation

End Sub

Public Function OnKeystroke(ByVal Wparam As Integer, ByVal Message As Integer, ByVal Lparam As Integer, ByVal Id As Integer) As
    OnKeystroke = True
End Function

Public Sub OnPopupMenuItem(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPopu

End Sub

Public Sub OnPopupMenuItemUpdate(ByVal Id As Integer, ByRef retval As Integer) Implements SolidWorks.Interop.swpublished.IProper

End Sub

Public Function OnPreview() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPreview
    OnPreview = True
End Function

Public Sub OnSliderPositionChanged(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IProper

End Sub

Public Sub OnSliderTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IProp

End Sub

Public Function OnSubmitSelection(ByVal Id As Integer, ByVal Selection As Object, ByVal SelType As Integer, ByRef ItemText As St
    OnSubmitSelection = True
End Function

Public Function OnTabClicked(ByVal Id As Integer) As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Hanc
    OnTabClicked = True
End Function

Public Sub OnUndo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnUndo

End Sub

Public Sub OnWhatsNew() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWhatsNew

End Sub

Function OnWindowFromHandleControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements SolidWorks.Interop
    System.Windows.Forms.MessageBox.Show(".NET control created")

```

```

        Return -1
    End Function

    Sub OnGainedFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnGainedFocus
    End Sub

    Sub OnListboxRMBUp(ByVal Id As Integer, ByVal PosX As Integer, ByVal PosY As Integer) Implements SolidWorks.Interop.swpublished.
    End Sub

    Sub OnLostFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnLostFocus
    End Sub

    Sub OnRedo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnRedo
    End Sub

    Sub OnNumberBoxTrackingCompleted(ByVal id As Integer, ByVal val As Double) Implements SolidWorks.Interop.swpublished.IPropertyMa
    End Sub
End Class

```

[Back to top](#)

```

'WPFControl.xaml
<UserControl x:Class="WPFControl"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Height="204" Width="314" Loaded="UserControl_Loaded">
    <Grid Height="44" Width="108">
        <ComboBox Text="WPF" Height="44" VerticalAlignment="Top">
            <ComboBoxItem Margin="2">
                <Ellipse Width="20" Height="20" Stroke="Black" />
            </ComboBoxItem>
            <ComboBoxItem Margin="2">
                <Path Stroke="Black" Data="M 0,0 L 20,0 L 10,20 Z" />
            </ComboBoxItem>
            <ComboBoxItem Margin="2">
                <Rectangle Stroke="Black" Width="20" Height="20" />
            </ComboBoxItem>
        </ComboBox>
    </Grid>
</UserControl>

```

[Back to top](#)

```

'WPFControl.xaml.vb
Imports System.Collections.Generic
Imports System.Text
Imports System.Windows
Imports System.Windows.Controls
Imports System.Windows.Data
Imports System.Windows.Documents
Imports System.Windows.Input
Imports System.Windows.Media
Imports System.Windows.Media.Imaging
Imports System.Windows.Navigation
Imports System.Windows.Shapes

```

```

Public Class WPFControl
    Inherits UserControl
    Sub New()
        InitializeComponent()
    End Sub

    Sub UserControl_Loaded(sender As Object, e As RoutedEventArgs)

    End Sub
End Class

```

[Back to top](#)

```

'UserControl1.vb
Imports System.Drawing
Imports System.Data
Imports System.Linq
Imports System.Text
Imports System.Windows.Forms

Partial Public Class UserControl1
    Inherits UserControl

    Public Sub New()
        InitializeComponent()
    End Sub

```

```

Private Sub textBox1_TextChanged_1(sender As Object, e As EventArgs) Handles textBox1.TextChanged

End Sub

Private Sub tabPage1_Click_1(sender As Object, e As EventArgs) Handles tabPage1.Click

End Sub

Private Sub tabPage2_Click_1(sender As Object, e As EventArgs) Handles tabPage2.Click

End Sub

Private Sub tabPage3_Click_1(sender As Object, e As EventArgs) Handles tabPage3.Click

End Sub

Private Sub radioButton1_CheckedChanged_1(sender As Object, e As EventArgs) Handles radioButton1.CheckedChanged

End Sub

Private Sub button1_Click_1(sender As Object, e As EventArgs) Handles button1.Click
    System.Windows.Forms.MessageBox.Show("Hello.")
End Sub

End Class

```

[Back to top](#)

```

'UserController1.Designer.vb
Partial Class UserController1
    ''' <summary>
    ''' Required designer variable.
    ''' </summary>
    Private components As System.ComponentModel.IContainer = Nothing

    ''' <summary>
    ''' Clean up any resources being used.
    ''' </summary>
    ''' <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
    Protected Overrides Sub Dispose(disposing As Boolean)
        If disposing AndAlso (components IsNot Nothing) Then
            components.Dispose()
        End If
        MyBase.Dispose(disposing)
    End Sub

#Region "Component Designer generated code"

    ''' <summary>
    ''' Required method for Designer support - do not modify
    ''' the contents of this method with the code editor.
    ''' </summary>
    Private Sub InitializeComponent()
        Me.radioButton1 = New System.Windows.Forms.RadioButton()
        Me.textBox1 = New System.Windows.Forms.TextBox()
        Me.button1 = New System.Windows.Forms.Button()
        Me.tabControl1 = New System.Windows.Forms.TabControl()
        Me.tabPage1 = New System.Windows.Forms.TabPage()
        Me.tabPage2 = New System.Windows.Forms.TabPage()
        Me.tabPage3 = New System.Windows.Forms.TabPage()
        Me.tabControl1.SuspendLayout()
        Me.SuspendLayout()
        '
        ' radioButton1
        '
        Me.radioButton1.Anchor = System.Windows.Forms.AnchorStyles.Left
        Me.radioButton1.AutoSize = True
        Me.radioButton1.Location = New System.Drawing.Point(23, 113)
        Me.radioButton1.Name = "radioButton1"
        Me.radioButton1.Size = New System.Drawing.Size(85, 17)
        Me.radioButton1.TabIndex = 5
        Me.radioButton1.TabStop = True
        Me.radioButton1.Text = "radioButton1"
        Me.radioButton1.UseVisualStyleBackColor = True
        '
        ' textBox1
        '
        Me.textBox1.Anchor = (System.Windows.Forms.AnchorStyles.Top Or System.Windows.Forms.AnchorStyles.Left) Or System.Windows.Forms.AnchorStyles.Right
        Me.textBox1.Location = New System.Drawing.Point(23, 87)
        Me.textBox1.Name = "textBox1"
        Me.textBox1.Size = New System.Drawing.Size(378, 20)
        Me.textBox1.TabIndex = 4
        '
        ' button1
        '
        Me.button1.Location = New System.Drawing.Point(23, 17)
        Me.button1.Name = "button1"

```

```

Me.button1.Size = New System.Drawing.Size(119, 49)
Me.button1.TabIndex = 3
Me.button1.Text = "Say Hello"
Me.button1.UseVisualStyleBackColor = True

'
' tabControl1
'
Me.tabControl1.Anchor = (System.Windows.Forms.AnchorStyles.Bottom Or System.Windows.Forms.AnchorStyles.Left) Or System.Wind
Me.tabControl1.Controls.Add(Me.TabPage1)
Me.tabControl1.Controls.Add(Me.TabPage2)
Me.tabControl1.Controls.Add(Me.TabPage3)
Me.tabControl1.Location = New System.Drawing.Point(23, 148)
Me.tabControl1.Multiline = True
Me.tabControl1.Name = "tabControl1"
Me.tabControl1.SelectedIndex = 0
Me.tabControl1.Size = New System.Drawing.Size(386, 69)
Me.tabControl1.TabIndex = 6
'
' TabPage1
'
Me.TabPage1.Location = New System.Drawing.Point(4, 22)
Me.TabPage1.Name = "TabPage1"
Me.TabPage1.Padding = New System.Windows.Forms.Padding(3)
Me.TabPage1.Size = New System.Drawing.Size(378, 43)
Me.TabPage1.TabIndex = 0
Me.TabPage1.Text = "TabPage1"
Me.TabPage1.UseVisualStyleBackColor = True

'
' TabPage2
'
Me.TabPage2.Location = New System.Drawing.Point(4, 22)
Me.TabPage2.Name = "TabPage2"
Me.TabPage2.Padding = New System.Windows.Forms.Padding(3)
Me.TabPage2.Size = New System.Drawing.Size(378, 43)
Me.TabPage2.TabIndex = 1
Me.TabPage2.Text = "TabPage2"
Me.TabPage2.UseVisualStyleBackColor = True

'
' TabPage3
'
Me.TabPage3.Location = New System.Drawing.Point(4, 22)
Me.TabPage3.Name = "TabPage3"
Me.TabPage3.Padding = New System.Windows.Forms.Padding(3)
Me.TabPage3.Size = New System.Drawing.Size(378, 43)
Me.TabPage3.TabIndex = 2
Me.TabPage3.Text = "TabPage3"
Me.TabPage3.UseVisualStyleBackColor = True

'
' UserControl1
'
Me.AutoScaleDimensions = New System.Drawing.SizeF(6.0F, 13.0F)
Me.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font
Me.Controls.Add(Me.tabControl1)
Me.Controls.Add(Me.radioButton1)
Me.Controls.Add(Me.textBox1)
Me.Controls.Add(Me.button1)
Me.Name = "UserControl1"
Me.Size = New System.Drawing.Size(427, 237)
Me.tabControl1.ResumeLayout(False)
Me.ResumeLayout(False)
Me.PerformLayout()

End Sub

#End Region

Private WithEvents radioButton1 As System.Windows.Forms.RadioButton
Private WithEvents textBox1 As System.Windows.Forms.TextBox
Private WithEvents button1 As System.Windows.Forms.Button
Private WithEvents tabControl1 As System.Windows.Forms.TabControl
Private WithEvents TabPage1 As System.Windows.Forms.TabPage
Private WithEvents TabPage2 As System.Windows.Forms.TabPage
Private WithEvents TabPage3 As System.Windows.Forms.TabPage
End Class

```

[Back to top](#)

```

'Form1.vb
Imports System.Collections.Generic
Imports System.ComponentModel
Imports System.Data
Imports System.Drawing
Imports System.Linq
Imports System.Text
Imports System.Windows.Forms

```

```

Partial Public Class Form1
    Inherits Form

```

```

Public Sub New()
    InitializeComponent()
End Sub

Private Sub button1_Click_1(sender As Object, e As EventArgs) Handles button1.Click
    System.Windows.Forms.MessageBox.Show("Hello.")
End Sub

Private Sub textBox1_TextChanged_1(sender As Object, e As EventArgs) Handles textBox1.TextChanged
End Sub

Private Sub comboBox1_SelectedIndexChanged_1(sender As Object, e As EventArgs) Handles comboBox1.SelectedIndexChanged
End Sub

Private Sub listBox1_SelectedIndexChanged_1(sender As Object, e As EventArgs) Handles listBox1.SelectedIndexChanged
End Sub

End Class

```

[Back to top](#)

'Form1.Designer.vb

```

Partial Class Form1
    ''' <summary>
    ''' Required designer variable.
    ''' </summary>
    Private components As System.ComponentModel.IContainer = Nothing

    ''' <summary>
    ''' Clean up any resources being used.
    ''' </summary>
    ''' <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
    Protected Overrides Sub Dispose(disposing As Boolean)
        If disposing AndAlso (components IsNot Nothing) Then
            components.Dispose()
        End If
        MyBase.Dispose(disposing)
    End Sub

    #Region "Windows Form Designer generated code"

        ''' <summary>
        ''' Required method for Designer support - do not modify
        ''' the contents of this method with the code editor.
        ''' </summary>
        Private Sub InitializeComponent()
            Me.listBox1 = New System.Windows.Forms.ListBox()
            Me.comboBox1 = New System.Windows.Forms.ComboBox()
            Me.textBox1 = New System.Windows.Forms.TextBox()
            Me.button1 = New System.Windows.Forms.Button()
            Me.SuspendLayout()
            '
            ' listBox1
            '
            Me.listBox1.Anchor = (System.Windows.Forms.AnchorStyles.Bottom Or System.Windows.Forms.AnchorStyles.Left) Or System.Windows.
            Me.listBox1.FormattingEnabled = True
            Me.listBox1.Items.AddRange(New Object() {"Help", "Whats New"})
            Me.listBox1.Location = New System.Drawing.Point(25, 160)
            Me.listBox1.Name = "listBox1"
            Me.listBox1.Size = New System.Drawing.Size(304, 43)
            Me.listBox1.TabIndex = 10
            '
            ' comboBox1
            '
            Me.comboBox1.DisplayMember = "Monday"
            Me.comboBox1.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList
            Me.comboBox1.FormattingEnabled = True
            Me.comboBox1.Items.AddRange(New Object() {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", _
            "Sunday"})
            Me.comboBox1.Location = New System.Drawing.Point(25, 122)
            Me.comboBox1.Name = "comboBox1"
            Me.comboBox1.Size = New System.Drawing.Size(140, 21)
            Me.comboBox1.TabIndex = 9
            Me.comboBox1.ValueMember = "Monday"
            '
            ' textBox1
            '
            Me.textBox1.Anchor = ((System.Windows.Forms.AnchorStyles.Top Or System.Windows.Forms.AnchorStyles.Bottom) Or System.Windows.
            Me.textBox1.Location = New System.Drawing.Point(25, 86)
            Me.textBox1.Name = "textBox1"
            Me.textBox1.Size = New System.Drawing.Size(321, 20)
            Me.textBox1.TabIndex = 8
        End Sub
    End Sub

```

```

' button1
'
Me.button1.Location = New System.Drawing.Point(25, 12)
Me.button1.Name = "button1"
Me.button1.Size = New System.Drawing.Size(140, 57)
Me.button1.TabIndex = 7
Me.button1.Text = "Say Hello"
Me.button1.UseVisualStyleBackColor = True

'
' Form1
'
Me.AutoScaleModeDimensions = New System.Drawing.SizeF(6.0F, 13.0F)
Me.AutoScaleModeMode = System.Windows.Forms.AutoScaleMode.Font
Me.ClientSize = New System.Drawing.Size(382, 215)
Me.ControlBox = False
Me.Controls.Add(Me.listBox1)
Me.Controls.Add(Me.comboBox1)
Me.Controls.Add(Me.textBox1)
Me.Controls.Add(Me.button1)
Me.FormBorderStyle = System.Windows.Forms.FormBorderStyle.None
Me.MaximizeBox = False
Me.MinimizeBox = False
Me.Name = "Form1"
Me.ShowIcon = False
Me.ShowInTaskbar = False
Me.Text = "Form1"
Me.ResumeLayout(False)
Me.PerformLayout()

End Sub

#End Region

Private WithEvents listBox1 As System.Windows.Forms.ListBox
Private WithEvents comboBox1 As System.Windows.Forms.ComboBox
Private WithEvents textBox1 As System.Windows.Forms.TextBox
Private WithEvents button1 As System.Windows.Forms.Button
End Class

```

[Back to top](#)

Add ActiveX Tab to FeatureManager Design Tree Example (C#)

This example shows how to add a tab to the FeatureManager design tree using three on-disk bitmap files for the tab's scaleable icon.

```
//-----
// Preconditions:
// 1. In the IDE, reference your ActiveX control file
//    a. Click Project > Add > Reference > Browse.
//    b. Browse to c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\SldUtils.
//    c. Select RichEditCtrl.ocx.
//    d. Click OK.
// 2. Verify that the specified part document exists.
// 3. Modify the namespace to match your project name.
// 4. Modify bitmapnames to reference your icon bitmaps in three sizes.
// 5. Review the ActiveX control reference and ensure that ctrlBtm is correct.
// 6. Search for RichEditCtrl.ocx in the registry and ensure that the second
//    argument to CreateFeatureMgrControl4 is correct.
//
// Postconditions:
// 1. Opens the part document and adds a new tab to the
//    FeatureManager design tree with an icon that uses the bitmap size
//    that fits the screen resolution.
//    NOTE: To add a tab to the FeatureManager tree,
//    you must set IModelViewManager::CreateFeatureMgrControl2's
//    WhichPane parameter to swFeatMgrPane_e.swFeatMgrPaneBottom.
// 2. Activates the new tab.
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace AddTabFeatureManagerDesignTreeCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModViewMgr = default(ModelViewManager);
            FeatMgrView swFeatMgrTabBtm = default(FeatMgrView);
            RICHEDITCTRLLib.RichEditCtrl ctrlBtm = default(RICHEDITCTRLLib.RichEditCtrl);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int activePane = 0;
            string[] bitmapnames = new String[3];

            bitmapnames[0] = "C:\\Users\\J4M\\Desktop\\icons\\mainicon_20.png";
            bitmapnames[1] = "C:\\Users\\J4M\\Desktop\\icons\\mainicon_32.png";
            bitmapnames[2] = "C:\\Users\\J4M\\Desktop\\icons\\mainicon_40.png";

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2020\\samples\\tutorial\\fillets\\knob.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModViewMgr = (ModelViewManager)swModel.ModelViewManager; // Create new tab

            swFeatMgrTabBtm = (FeatMgrView)swModViewMgr.CreateFeatureMgrControl4(bitmapnames, "GTSWRICHEDITCTRL.RichEditCtrlCtrl.1", "",
            "Tab Tooltip", (int)swFeatMgrPane_e.swFeatMgrPaneBottom);
            ctrlBtm = (RICHEDITCTRLLib.RichEditCtrl)swFeatMgrTabBtm.GetControl();
            activePane = swFeatMgrTabBtm.ActivateView();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Add ActiveX Tab to FeatureManager Design Tree Example (VBA)

This example shows how to add a tab to the FeatureManager design tree using three on-disk bitmap files for the tab's scaleable icon.

```
'-----
' Preconditions:
' 1. In the IDE, reference the ActiveX control file
'   a. Click Tools > References > Browse.
'   b. Browse to c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\SldUtils.
'   c. Select RichEditCtrl.ocx.
'   d. Click Open.
'   e. Click OK.
' 2. Insert the ActiveX component
'   Click Insert > Components > RichEditCtrl ActiveX Control module > OK.
' 3. Verify that the specified part document exists.
' 4. Modify the paths in bitmapnames to reference your tab icon's bitmap files in three sizes.
' 5. Review the ActiveX control reference and ensure that the ctrlBtm declaration is correct.
' 6. Search for RichEditCtrl.ocx in the registry and ensure that the second
'    argument of the CreateFeatureMgrControl4 call is correct.
'
' Postconditions:
' 1. Opens the part document and adds a new tab
'    to the FeatureManager design tree with an icon that uses the bitmap size that
'    fits the screen resolution.
'    NOTE: To add a tab to the FeatureManager tree,
'    you must set IModelViewManager::CreateFeatureMgrControl4's
'    WhichPane parameter to swFeatMgrPane_e.swFeatMgrPaneBottom.
' 2. Activates the new tab and displays the ActiveX control.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModViewMgr As SldWorks.ModelViewManager
    Dim swFeatMgrTabBtm As SldWorks.FeatMgrView
    Dim ctrlBtm As RICHEDITCTRLLib.RichEditCtrl
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long
    Dim activePane As Long
    Dim bitmapnames(2) As String

    Set swApp = CreateObject("SldWorks.Application")
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\fillets\knob.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModViewMgr = swModel.ModelViewManager

    bitmapnames(0) = "C:\Users\J4M\Desktop\icons\mainicon_20.png"
    bitmapnames(1) = "C:\Users\J4M\Desktop\icons\mainicon_32.png"
    bitmapnames(2) = "C:\Users\J4M\Desktop\icons\mainicon_40.png"

    ' Create and activate the new tab
    Set swFeatMgrTabBtm = swModViewMgr.CreateFeatureMgrControl4(bitmapnames, "GTSWRICHEDITCTRL.RichEditCtrlCtrl.1", "", "Tab ToolTip",
swFeatMgrPaneBottom)
    Set ctrlBtm = swFeatMgrTabBtm.GetControl
    activePane = swFeatMgrTabBtm.ActivateView

End Sub
```

Add ActiveX Tab to FeatureManager Design Tree Example (VB.NET)

This example shows how to add a tab to the FeatureManager design tree using three on-disk bitmap files for the tab's scaleable icon.

```

'-----
' Preconditions:
' 1. In the IDE, reference your ActiveX control file
'    a. Click Project > Add > Reference > Browse.
'    b. Browse to c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\SldUtils.
'    c. Select RichEditCtrl.ocx.
'    d. Click OK.
' 2. Verify that the specified part document exists.
' 3. Modify bitmapnames to reference your icon bitmaps in three sizes.
' 4. Review the ActiveX control reference and ensure that ctrlBtm is correct.
' 5. Search for RichEditCtrl.ocx in the registry and ensure that the second
'    argument to CreateFeatureMgrControl4 is correct.
'
' Postconditions:
' 1. Opens the part document and adds a new tab to the
'    FeatureManager design tree with an icon that uses the bitmap size
'    that fits the screen resolution.
'    NOTE: To add a tab to the FeatureManager tree,
'    you must set IModelViewManager::CreateFeatureMgrControl2's
'    WhichPane parameter to swFeatMgrPane_e.swFeatMgrPaneBottom.
' 2. Activates the new tab.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModViewMgr As ModelViewManager
        Dim swFeatMgrTabBtm As FeatMgrView
        Dim ctrlBtm As RICHEDITCTRLLib.RichEditCtrl
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim activePane As Integer
        Dim BitmapFileNames(2) As String

        BitmapFileNames(0) = "C:\Users\J4M\Desktop\icons\mainicon_20.png"
        BitmapFileNames(1) = "C:\Users\J4M\Desktop\icons\mainicon_32.png"
        BitmapFileNames(2) = "C:\Users\J4M\Desktop\icons\mainicon_40.png"

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\fillets\knob.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)

        swModViewMgr = swModel.ModelViewManager

        ' Create new tab

        swFeatMgrTabBtm = swModViewMgr.CreateFeatureMgrControl4(BitmapFileNames, "GTSWRICHEDITCTRL.RichEditCtrlCtrl.1", "", "Tab
ToolTip", swFeatMgrPane_e.swFeatMgrPaneBottom)
        ctrlBtm = swFeatMgrTabBtm.GetControl
        activePane = swFeatMgrTabBtm.ActivateView

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Add ActiveX Tabs to FeatureManager Design Tree Example (C#)

This example shows how to add tabs to a split FeatureManager design tree using an on-disk bitmap file for the tabs' icons.

```
//-----
// Preconditions:
// 1. In the IDE, reference your ActiveX control file
//    (click Project > Add Reference > Browse and browse
//    to the folder where the ActiveX control resides and click
//    the ActiveX control file > OK).
// 2. Verify that the specified part document and bitmap exist.
// 3. Replace activex_control_component_declaration and
//    activex_control_CLSID_or_ProgID with your ActiveX control's
//    information.
//
// Postconditions:
// 1. Opens the part document and splits the FeatureManager
//    design tree.
// 2. Adds a new tab to each FeatureManager design tree.
// 3. Activates the new tab on the top FeatureManager
//    design tree.
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace AddTabsFeatureManagerDesignTreeCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            const string iconSmall = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\data\\user macro icons\\button.bmp";

            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModViewMgr = default(ModelViewManager);
            FeatMgrView swFeatMgrTabTop = default(FeatMgrView);
            FeatMgrView swFeatMgrTabBtm = default(FeatMgrView);
            activex_control_component_declaration ctrlTop = default(activex_control_component_declaration);
            activex_control_component_declaration ctrlBtm = default(activex_control_component_declaration);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int activePane = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\fillets\\knob.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            swModViewMgr = (ModelViewManager)swModel.ModelViewManager;

            // Split the FeatureManager design tree;
            // the original is displayed below the copy
            swModel.FeatureManagerSplitterPosition = 0.5;

            // New tab on copy of FeatureManager design tree
            swFeatMgrTabBtm = (FeatMgrView)swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "",
            "Bottom tab Tooltip", (int)swFeatMgrPane_e.swFeatMgrPaneBottom);
            ctrlBtm = (activex_control_component_declaration)swFeatMgrTabBtm.GetControl();

            // New tab on original of FeatureManager design tree
            swFeatMgrTabTop = (FeatMgrView)swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "",
            "Top tab Tooltip", (int)swFeatMgrPane_e.swFeatMgrPaneTop);
            ctrlTop = (activex_control_component_declaration)swFeatMgrTabTop.GetControl();
            activePane = swFeatMgrTabTop.ActivateView();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Add ActiveX Tabs to FeatureManager Design Tree Example (VBA)

This example shows how to add tabs to a split FeatureManager design tree using an on-disk bitmap file for the tabs' icon.

```
'-----
' Preconditions:
' 1. In the IDE:
'   a. Reference your ActiveX control file (click Tools >
'      References > Browse > browse to the folder where the
'      ActiveX control file resides and click the ActiveX
'      control file > Open > OK).
'   a. Insert the ActiveX component (click Insert > Components >
'      ActiveX component's check box > OK).
' 2. Verify that the specified part document and bitmap exist.
' 3. Replace activex_control_component_declaration and
'    activex_control_CLSID_or_ProgID with your ActiveX control's
'    information.
'
' Postconditions:
' 1. Opens the part document and splits the FeatureManager
'    design tree.
' 2. Adds a new tab to each FeatureManager design tree.
' 3. Activates the new tab on the top FeatureManager design tree.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

```
Sub main()

    Const iconSmall As String = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\user macro icons\button.bmp"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModViewMgr As SldWorks.ModelViewManager
    Dim swFeatMgrTabTop As SldWorks.FeatMgrView
    Dim swFeatMgrTabBtm As SldWorks.FeatMgrView
    Dim ctrlTop As activex_control_component_declaration
    Dim ctrlBtm As activex_control_component_declaration
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long
    Dim activePane As Long

    Set swApp = CreateObject("SldWorks.Application")

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

    Set swModViewMgr = swModel.ModelViewManager

    ' Split the FeatureManager design tree;
    ' the original is displayed below the copy
    swModel.FeatureManagerSplitterPosition = 0.5

    ' New tab on copy of FeatureManager design tree
    Set swFeatMgrTabTop = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Top tab ToolTip", swFeatMgrPaneTop)
    Set ctrlTop = swFeatMgrTabTop.GetControl

    ' New tab on original FeatureManager design tree
    Set swFeatMgrTabBtm = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Bottom tab ToolTip",
swFeatMgrPaneBottom)
    Set ctrlBtm = swFeatMgrTabBtm.GetControl
    activePane = swFeatMgrTabTop.ActivateView

End Sub
```

Add ActiveX Tabs to FeatureManager Design Tree Example (VB.NET)

This example shows how to add tabs to a split FeatureManager design tree using an on-disk bitmap file for the tabs' icons.

```

'-----
' Preconditions:
' 1. In the IDE, reference your ActiveX control file
'    (click Project > Add Reference > Browse and browse
'    to the folder where the ActiveX control resides and click
'    the ActiveX control file > OK).
' 2. Verify that the specified part document and bitmap exist.
' 3. Replace activex_control_component_declaration and
'    activex_control_CLSID_or_ProgID with your ActiveX control's
'    information.
'
' Postconditions:
' 1. Opens the part document and splits the FeatureManager
'    design tree.
' 2. Adds a new tab to each FeatureManager design tree.
' 3. Activates the new tab on the top of the FeatureManager
'    design tree.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst

Partial Class SolidWorksMacro

    Public Sub Main()

        Const iconSmall As String = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\user macro icons\button.bmp"

        Dim swModel As ModelDoc2
        Dim swModViewMgr As ModelViewManager
        Dim swFeatMgrTabTop As FeatMgrView
        Dim swFeatMgrTabBtm As FeatMgrView
        Dim ctrlTop As activex_control_component_declaration
        Dim ctrlBtm As activex_control_component_declaration
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim activePane As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
        warnings)

        swModViewMgr = swModel.ModelViewManager

        ' Split the FeatureManager design tree;
        ' the original is displayed below the copy
        swModel.FeatureManager.SplitterPosition = 0.5

        ' New tab on original of FeatureManager design tree
        swFeatMgrTabBtm = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Bottom tab
ToolTip", swFeatMgrPane_e.swFeatMgrPaneBottom)
        ctrlBtm = swFeatMgrTabBtm.GetControl

        ' New tab on copy of FeatureManager design tree
        swFeatMgrTabTop = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Top tab ToolTip",
        swFeatMgrPane_e.swFeatMgrPaneBottom)
        ctrlTop = swFeatMgrTabTop.GetControl
        activePane = swFeatMgrTabTop.ActivateView

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Add ActiveX Tabs to Model View Example (C#)

This example shows how to add two ActiveX tabs to a model view.

```
//-----
// Preconditions:
// 1. In the IDE, reference your ActiveX control file
//    (click Project > Add Reference > Browse and browse
//    to the folder where the ActiveX control resides and click
//    the ActiveX control file > OK).
// 2. Verify that the specified part document exists.
// 3. Replace activex_control_component_declaration and
//    activex_control_CLSID_or_ProgID with your ActiveX control's
//    information.
//
// Postconditions:
// 1. Opens the part document and adds two new tabs to the model view.
// 2. Activates the model view.
// 3. To verify the ActiveX controls on the new tabs, click each tab.
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using Microsoft.VisualBasic;

namespace AddControlModelViewManagerCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            const string calTabName1 = "Tab 1";
            const string calTabName2 = "Tab 2";

            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModViewMgr = default(ModelViewManager);
            activex_control_component_declaration calCtrl1 = default(activex_control_component_declaration);
            activex_control_component_declaration calCtrl2 = default(activex_control_component_declaration);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool status = false;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\fillets\\knob.sldprt";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "",
ref errors, ref warnings);
            swModViewMgr = swModel.ModelViewManager;

            // Add the tabs
            calCtrl1 = (activex_control_component_declaration)swModViewMgr.AddControl(calTabName1, "activex_control_CLSID_or_ProgID",
            "");
            calCtrl2 = (activex_control_component_declaration)swModViewMgr.AddControl(calTabName2, "activex_control_CLSID_or_ProgID",
            "");

            status = swModViewMgr.ActivateControlTab(calTabName1);
            status = swModViewMgr.IsControlTabActive(calTabName1);
            status = swModViewMgr.IsModelTabActive();

            // Switch back to model view
            status = swModViewMgr.ActivateModelTab();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Add ActiveX Tabs to Model View Example (VBA)

This example shows how to add two ActiveX tabs to a model view.

```
'-----
' Preconditions:
' 1. In the IDE:
'   a. Reference your ActiveX control file (click Tools >
'      References > Browse > browse to the folder where the
'      ActiveX control file resides and click the ActiveX
'      control file > Open > OK).
'   a. Insert the ActiveX component (click Insert > Components >
'      ActiveX component's check box > OK).
' 2. Verify that the specified part document exists.
' 3. Replace activex_control_component_declaration and
'    activex_control_CLSID_or_ProgID with your ActiveX control's
'    information.
'
' Postconditions:
' 1. Opens the part document and adds two new tabs to the model view.
' 2. Activates the model view.
' 3. To verify the ActiveX controls on the new tabs,
'    click each tab.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

Sub Main()

```
    Const calTabName1 As String = "Tab 1"
    Const calTabName2 As String = "Tab 2"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModViewMgr As SldWorks.ModelViewManager
    Dim calCtrl1 As activex_control_component_declaration
    Dim calCtrl2 As activex_control_component_declaration
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long
    Dim status As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModViewMgr = swModel.ModelViewManager

    ' Add the ActiveX control tabs
    Set calCtrl1 = swModViewMgr.AddControl(calTabName1, "activex_control_CLSID_or_ProgID", "")
    Set calCtrl2 = swModViewMgr.AddControl(calTabName2, "activex_control_CLSID_or_ProgID", "")

    status = swModViewMgr.ActivateControlTab(calTabName1)
    status = swModViewMgr.IsControlTabActive(calTabName1)
    status = swModViewMgr.IsModelTabActive

    ' Switch back to model view
    status = swModViewMgr.ActivateModelTab
```

End Sub

Add ActiveX Tabs to Model View Example (VB.NET)

This example shows how to add two ActiveX tabs to a model view.

```

'-----
' Preconditions:
' 1. In the IDE, reference your ActiveX control file
'    (click Project > Add Reference > Browse and browse
'    to the folder where the ActiveX control resides and click
'    the ActiveX control file > OK).
' 2. Verify that the specified part document exists.
' 3. Replace activex_control_component_declaration and
'    activex_control_CLSID_or_ProgID with your ActiveX control's
'    information.
'
' Postconditions:
' 1. Opens the part document and adds two new tabs to the model view.
' 2. Activates the model view.
' 3. To verify the ActiveX controls on the new tabs,
'    interactively click tab.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Const calTabName1 As String = "Tab 1"
        Const calTabName2 As String = "Tab 2"

        Dim swModel As ModelDoc2
        Dim swModViewMgr As ModelViewManager
        Dim calCtrl1 As activex_control_component_declaration
        Dim calCtrl2 As activex_control_component_declaration
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim status As Boolean

        swApp = CreateObject("SldWorks.Application")
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)
        swModViewMgr = swModel.ModelViewManager

        ' Add the ActiveX control tabs
        calCtrl1 = swModViewMgr.AddControl(calTabName1, "activex_control_CLSID_or_ProgID", "")
        calCtrl2 = swModViewMgr.AddControl(calTabName2, "activex_control_CLSID_or_ProgID", "")

        status = swModViewMgr.ActivateControlTab(calTabName1)
        status = swModViewMgr.IsControlTabActive(calTabName1)
        status = swModViewMgr.IsModelTabActive

        ' Switch back to model view
        status = swModViewMgr.ActivateModelTab

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Add Along X Dimension to 3D Sketch Example (C#)

This example shows how to add a display dimension along the x axis in a 3D sketch.

```

//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Inserts a 3D sketch.
// 3. Click the green check mark in the Modify dimension dialog
//    (If you don't see the dialog, look for it behind other open windows).
// 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a
//    corner rectangle.
// 5. Displays the display dimension of 84.46 mm on the x axis while the
//    sketch is in edit mode.
// 6. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            DisplayDimension myDisplayDim = default(DisplayDimension);
            bool boolstatus = false;
            int longstatus = 0;

            longstatus = swApp.ResetUntitledCount(0, 0, 0);
            Part = (ModelDoc2)swApp.NewDocument("C:\\Documents and Settings\\All Users\\Application Data\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prt
            dot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.SketchManager.Insert3DSketch(true);
            object vSkLines = null;
            vSkLines = (object)Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0
            .03);

            boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.ClearSelection2(true);

            object pointArray = null;
            double[] points = new double[12];
            points[0] = 0;
            points[1] = -0.03591009660795;
            points[2] = 0.04608246573503;
            points[3] = 0;
            points[4] = 0.0147420284178;
            points[5] = 0.005170989573514;
            points[6] = 0;
            points[7] = -0.006478053228363;
            points[8] = -0.04282131900055;
            points[9] = 0;
            points[10] = -0.02294509596464;
            points[11] = -0.09396066420243;
            pointArray = (object)points;

            SketchSegment skSegment = default(SketchSegment);
            skSegment = (SketchSegment)Part.SketchManager.CreateSpline2((pointArray), true);
            Part.SketchManager.InsertSketch(true);

            boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            Part.EditSketch();
            boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, true, 0, null, 0);
            myDisplayDim = (DisplayDimension)Part.SketchManager.AddAlongXDimension(0.05, -0.091, 0.001);
            Part.ClearSelection2(true);

            Part.ViewZoomtofit2();
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Add Along X Dimension to 3D Sketch Example (VBA)

This example shows how to add a display dimension along the x axis in a 3D sketch.

```
' -----
' Preconditions: Verify that the specified part template exists.
'
```

```
' Postconditions:
' 1. Opens a new part document.
' 2. Inserts a 3D sketch.
' 3. Click the green check mark in the Modify dimension dialog
'    (If you don't see the dialog, look for it behind other open windows).
' 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a
'    corner rectangle.
' 5. Displays the display dimension of 84.46 mm on the x axis while the
'    sketch is in edit mode.
' 6. Examine the graphics area.
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim longstatus As Long
Sub main()
```

```
    Set swApp = Application.SldWorks
    longstatus = swApp.ResetUntitledCount(0, 0, 0)
    Set Part = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set Part = swApp.ActiveDoc
```

```
    Part.SketchManager.Insert3DSketch True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0.03)
    boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
```

```
    Dim pointArray As Variant
    Dim points() As Double
    ReDim points(0 To 11) As Double
    points(0) = 0
    points(1) = -0.03591009660795
    points(2) = 0.04608246573503
    points(3) = 0
    points(4) = 0.0147420284178
    points(5) = 0.005170989573514
    points(6) = 0
    points(7) = -0.006478053228363
    points(8) = -0.04282131900055
    points(9) = 0
    points(10) = -0.02294509596464
    points(11) = -0.09396066420243
    pointArray = points
```

```
    Dim skSegment As SldWorks.SketchSegment
    Set skSegment = Part.SketchManager.CreateSpline2((pointArray), True)
    Part.SketchManager.InsertSketch True
```

```
    boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditSketch
    boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, True, 0, Nothing, 0)
    Set myDisplayDim = Part.SketchManager.AddAlongXDimension(0.05, -0.091, 0.001)
    Part.ClearSelection2 True
```

```
    Part.ViewZoomtofit2
```

```
End Sub
```

Add Along X Dimension to 3D Sketch Example (VB.NET)

This example shows how to add a display dimension along the x axis in a 3D sketch.

```
'-----
' Preconditions: Verify that the specified part template exists.
```

```
' Postconditions:
' 1. Opens a new part document.
' 2. Inserts a 3D sketch.
' 3. Click the green check mark in the Modify dimension dialog
'    (If you don't see the dialog, look for it behind other open windows).
' 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a
'    corner rectangle.
' 5. Displays the display dimension of 84.46 mm on the x axis while the
'    sketch is in edit mode.
' 6. Examine the graphics area.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim Part As ModelDoc2
```

```
        Dim myDisplayDim As DisplayDimension
```

```
        Dim boolstatus As Boolean
```

```
        Dim longstatus As Integer
```

```
        longstatus = swApp.ResetUntitledCount(0, 0, 0)
```

```
        Part = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prt", 0, 0, 0)
```

```
        swApp.ActivateDoc2("Part1", False, longstatus)
```

```
        Part = swApp.ActiveDoc
```

```
        Part.SketchManager.Insert3DSketch(True)
```

```
        Dim vSkLines As Object
```

```
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0.03)
```

```
        boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Part.ClearSelection2(True)
```

```
        Dim pointArray As Object
```

```
        Dim points(11) As Double
```

```
        points(0) = 0
```

```
        points(1) = -0.03591009660795
```

```
        points(2) = 0.04608246573503
```

```
        points(3) = 0
```

```
        points(4) = 0.0147420284178
```

```
        points(5) = 0.005170989573514
```

```
        points(6) = 0
```

```
        points(7) = -0.006478053228363
```

```
        points(8) = -0.04282131900055
```

```
        points(9) = 0
```

```
        points(10) = -0.02294509596464
```

```
        points(11) = -0.09396066420243
```

```
        pointArray = points
```

```
        Dim skSegment As SketchSegment
```

```
        skSegment = Part.SketchManager.CreateSpline2((pointArray), True)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Part.EditSketch()
```

```
        boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, False, 0, Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, True, 0, Nothing, 0)
```

```
        myDisplayDim = Part.SketchManager.AddAlongXDimension(0.05, -0.091, 0.001)
```

```
        Part.ClearSelection2(True)
```

```
        Part.ViewZoomtofit2()
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Add Along Y Dimension to 3D Sketch Example (C#)

This example shows how to add a display dimension along the y axis in a 3D sketch.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a 3D sketch.
// 3. Click the green check mark in the Modify dimension dialog
//    (If you don't see the dialog, look for it behind other open windows).
// 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a
//    corner rectangle.
// 5. Displays the display dimension of 63.24 mm on the y axis starting at
//    (-0.1, 0, 0.01111142101618) while the sketch is in edit mode.
// 6. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            DisplayDimension myDisplayDim = default(DisplayDimension);
            bool boolstatus = false;
            int longstatus = 0;

            longstatus = swApp.ResetUntitledCount(0, 0, 0);
            Part = (ModelDoc2)swApp.NewDocument("C:\\Documents and Settings\\All Users\\Application Data\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prt
            dot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.SketchManager.Insert3DSketch(true);
            object vSkLines = null;
            vSkLines = (object)Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0
            .03);

            boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.ClearSelection2(true);

            object pointArray = null;
            double[] points = new double[12];
            points[0] = 0;
            points[1] = -0.03591009660795;
            points[2] = 0.04608246573503;
            points[3] = 0;
            points[4] = 0.0147420284178;
            points[5] = 0.005170989573514;
            points[6] = 0;
            points[7] = -0.006478053228363;
            points[8] = -0.04282131900055;
            points[9] = 0;
            points[10] = -0.02294509596464;
            points[11] = -0.09396066420243;
            pointArray = (object) points;

            SketchSegment skSegment = default(SketchSegment);
            skSegment = (SketchSegment)Part.SketchManager.CreateSpline2((pointArray), true);
            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            Part.EditSketch();
            boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, true, 0, null, 0);
            myDisplayDim = (DisplayDimension)Part.SketchManager.AddAlongYDimension(-0.1, 0, 0.01111142101618);
            Part.ClearSelection2(true);

            Part.ViewZoomToFit2();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Add Along Y Dimension to 3D Sketch Example (VBA)

This example shows how to add a display dimension along the y axis in a 3D sketch.

```
' -----
' Preconditions: Verify that the specified part template exists.
'
```

```
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a 3D sketch.
' 3. Click the green check mark in the Modify dimension dialog
'    (If you don't see the dialog, look for it behind other open windows).
' 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a
'    corner rectangle.
' 5. Displays the display dimension of 63.24 mm on the y axis starting at
'    (-0.1, 0, 0.01111142101618) while the sketch is in edit mode.
' 6. Examine the graphics area.
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim longstatus As Long
```

Sub main()

```
Set swApp = Application.SldWorks
longstatus = swApp.ResetUntitledCount(0, 0, 0)
Set Part = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
swApp.ActivateDoc2 "Part1", False, longstatus
Set Part = swApp.ActiveDoc
```

```
Part.SketchManager.Insert3DSketch True
Dim vSkLines As Variant
vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0.03)
boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
Part.ClearSelection2 True
```

```
Dim pointArray As Variant
Dim points() As Double
ReDim points(0 To 11) As Double
points(0) = 0
points(1) = -0.03591009660795
points(2) = 0.04608246573503
points(3) = 0
points(4) = 0.0147420284178
points(5) = 0.005170989573514
points(6) = 0
points(7) = -0.006478053228363
points(8) = -0.04282131900055
points(9) = 0
points(10) = -0.02294509596464
points(11) = -0.09396066420243
pointArray = points
```

```
Dim skSegment As SldWorks.SketchSegment
Set skSegment = Part.SketchManager.CreateSpline2((pointArray), True)
Part.SketchManager.InsertSketch True
boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
Part.EditSketch
```

```
boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, False, 0, Nothing, 0)
boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, True, 0, Nothing, 0)
Set myDisplayDim = Part.SketchManager.AddAlongYDimension(-0.1, 0, 0.01111142101618)
Part.ClearSelection2 True
```

Part.ViewZoomtofit2

End Sub

Add Along Y Dimension to 3D Sketch Example (VB.NET)

This example shows how to add a display dimension along the y axis in a 3D sketch.

Preconditions: Verify that the specified part template exists.

Postconditions:

1. Opens a new part document.
2. Creates a 3D sketch.
3. Click the green check mark in the Modify dimension dialog (If you don't see the dialog, look for it behind other open windows).
4. Puts **3DSketch1** in edit mode; **3DSketch1** contains a spline and a corner rectangle.
5. Displays the display dimension of **63.24 mm** on the y axis starting at (-0.1, 0, 0.01111142101618) while the sketch is in edit mode.
6. Examine the graphics area.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()

        Dim Part As ModelDoc2
        Dim myDisplayDim As DisplayDimension
        Dim boolstatus As Boolean
        Dim longstatus As Integer

        longstatus = swApp.ResetUntitledCount(0, 0, 0)
        Part = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prt", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        Part = swApp.ActiveDoc

        Part.SketchManager.Insert3DSketch(True)
        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0.03)
        boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        Part.ClearSelection2(True)

        Dim pointArray As Object
        Dim points(11) As Double
        points(0) = 0
        points(1) = -0.03591009660795
        points(2) = 0.04608246573503
        points(3) = 0
        points(4) = 0.0147420284178
        points(5) = 0.005170989573514
        points(6) = 0
        points(7) = -0.006478053228363
        points(8) = -0.04282131900055
        points(9) = 0
        points(10) = -0.02294509596464
        points(11) = -0.09396066420243
        pointArray = points

        Dim skSegment As SketchSegment
        skSegment = Part.SketchManager.CreateSpline2((pointArray), True)
        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        Part.EditSketch()
        boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, True, 0, Nothing, 0)
        myDisplayDim = Part.SketchManager.AddAlongYDimension(-0.1, 0, 0.01111142101618)
        Part.ClearSelection2(True)

        Part.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
```

```
End Class
```

Add Along Z Dimension to 3D Sketch Example (C#)

This example shows how to add a display dimension along the z axis in a 3D sketch.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a 3D sketch.
// 3. Click the green check mark in the Modify dimension dialog
//    (If you don't see the dialog, look for it behind other open windows).
// 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a corner
//    rectangle.
// 5. Displays the display dimension of 64.81 mm on the z axis starting at
//    (-0.03841894197919, -0.03273212874668, 0.042510877252)
//    while the sketch is in edit mode.
// 6. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            DisplayDimension myDisplayDim = default(DisplayDimension);
            bool boolstatus = false;
            int longstatus = 0;

            longstatus = swApp.ResetUntitledCount(0, 0, 0);
            Part = (ModelDoc2)swApp.NewDocument("C:\\Documents and Settings\\All Users\\Application Data\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.prt
            dot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            Part.SketchManager.Insert3DSketch(true);
            object vSkLines = null;
            vSkLines = (object)Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0
            .03);

            boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.ClearSelection2(true);

            object pointArray = null;
            double[] points = new double[12];
            points[0] = 0;
            points[1] = -0.03591009660795;
            points[2] = 0.04608246573503;
            points[3] = 0;
            points[4] = 0.0147420284178;
            points[5] = 0.005170989573514;
            points[6] = 0;
            points[7] = -0.006478053228363;
            points[8] = -0.04282131900055;
            points[9] = 0;
            points[10] = -0.02294509596464;
            points[11] = -0.09396066420243;
            pointArray = (object) points;

            SketchSegment skSegment = default(SketchSegment);
            skSegment = (SketchSegment)Part.SketchManager.CreateSpline2((pointArray), true);
            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            Part.EditSketch();
            boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, true, 0, null, 0);
            myDisplayDim = (DisplayDimension)Part.SketchManager.AddAlongZDimension(-0.03841894197919, -0.03273212874668, 0.042510877252);
            Part.ClearSelection2(true);

            Part.ViewZoomToFit2();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Add Along Z Dimension to 3D Sketch Example (VBA)

This example shows how to add a display dimension along the z axis in a 3D sketch.

```
' -----
' Preconditions: Verify that the specified part template exists.
'
```

```
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a 3D sketch.
' 3. Click the green check mark in the Modify dimension dialog
'    (If you don't see the dialog, look for it behind other open windows).
' 4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a corner
'    rectangle.
' 5. Displays the display dimension of 64.81 mm on the z axis starting at
'    (-0.03841894197919, -0.03273212874668, 0.042510877252)
'    while the sketch is in edit mode.
' 6. Examine the graphics area.
' -----
```

```
Option Explicit
```

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean
Dim longstatus As Long
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
    longstatus = swApp.ResetUntitledCount(0, 0, 0)
    Set Part = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set Part = swApp.ActiveDoc
```

```
    Part.SketchManager.Insert3DSketch True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0.03)
    boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
```

```
    Dim pointArray As Variant
    Dim points() As Double
    ReDim points(0 To 11) As Double
    points(0) = 0
    points(1) = -0.03591009660795
    points(2) = 0.04608246573503
    points(3) = 0
    points(4) = 0.0147420284178
    points(5) = 0.005170989573514
    points(6) = 0
    points(7) = -0.006478053228363
    points(8) = -0.04282131900055
    points(9) = 0
    points(10) = -0.02294509596464
    points(11) = -0.09396066420243
    pointArray = points
```

```
    Dim skSegment As SldWorks.SketchSegment
    Set skSegment = Part.SketchManager.CreateSpline2((pointArray), True)
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditSketch
    boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, True, 0, Nothing, 0)
    Set myDisplayDim = Part.SketchManager.AddAlongZDimension(-0.03841894197919, -0.03273212874668, 0.042510877252)
    Part.ClearSelection2 True
```

```
    Part.ViewZoomtofit2
```

```
End Sub
```

Add Along Z Dimension to 3D Sketch Example (VB.NET)

This example shows how to add a display dimension along the z axis in a 3D sketch.

```

-----
Preconditions: Verify that the specified part template exists.
-----
Postconditions:
1. Opens a new part document.
2. Creates a 3D sketch.
3. Click the green check mark in the Modify dimension dialog
   (If you don't see the dialog, look for it behind other open windows).
4. Puts 3DSketch1 in edit mode; 3DSketch1 contains a spline and a corner
   rectangle.
5. Displays the display dimension of 64.81 mm on the z axis starting at
   (-0.03841894197919, -0.03273212874668, 0.042510877252)
   while the sketch is in edit mode.
6. Examine the graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim Part As ModelDoc2
        Dim myDisplayDim As DisplayDimension
        Dim boolstatus As Boolean
        Dim longstatus As Integer

        longstatus = swApp.ResetUntitledCount(0, 0, 0)
        Part = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prt", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        Part = swApp.ActiveDoc

        Part.SketchManager.Insert3DSketch(True)
        Dim vSkLines As Object
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.05171778666374, 0.01933785938058, 0.03, 0.08445537697179, -0.04142795937025, -0.03)
        boolstatus = Part.Extension.SelectByID2("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        Part.ClearSelection2(True)

        Dim pointArray As Object
        Dim points(11) As Double
        points(0) = 0
        points(1) = -0.03591009660795
        points(2) = 0.04608246573503
        points(3) = 0
        points(4) = 0.0147420284178
        points(5) = 0.005170989573514
        points(6) = 0
        points(7) = -0.006478053228363
        points(8) = -0.04282131900055
        points(9) = 0
        points(10) = -0.02294509596464
        points(11) = -0.09396066420243
        pointArray = points

        Dim skSegment As SketchSegment
        skSegment = Part.SketchManager.CreateSpline2((pointArray), True)
        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        Part.EditSketch()
        boolstatus = Part.Extension.SelectByID2("Point5", "SKETCHPOINT", 0, -0.03591009660795, 0.04608246573503, False, 0, Nothing, 0)
        boolstatus = Part.Extension.SelectByID2("Point4", "SKETCHPOINT", 0.08445537697179, 0.02732744880518, -0.01872625210654, True, 0, Nothing, 0)
        myDisplayDim = Part.SketchManager.AddAlongZDimension(-0.03841894197919, -0.03273212874668, 0.042510877252)
        Part.ClearSelection2(True)

        Part.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Add and Delete Appearances from Specific Display States Example (C#)

This example shows how to add an appearance to and delete an appearance from specific display states.

```
//-----
// Preconditions:
// 1. Specified model exists.
// 2. Specified appearance exists.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Creates Display State 2 and Display State 3 for the active
//    configuration.
// 2. Applies specified appearance to all display states of the active
//    configuration.
// 3. Press F5.
// 4. Deletes specified appearance from all display states of the active
//    configuration.
// 5. Press F5.
// 6. Closes document.
//-----

using
Microsoft.VisualBasic;
using

System;
using

System.Collections;
using

System.Collections.Generic;
using

System.Data;
using

System.Diagnostics;
using

SolidWorks.Interop.sldworks;
using

SolidWorks.Interop.swconst;
using

System.Runtime.InteropServices;
namespace
RenderMaterials_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Configuration swConfig;
        ModelDocExtension swModelDocExt;
        Entity swEntity;
        SelectionMgr swSelMgr;
        RenderMaterial swRenderMaterial;
        object[] displayStateNames;
        bool status;
        string modelName;
        string materialName;
        int errors;
        int warnings;
        int nbrDisplayStates;
        int i;
        int k;
        int nbrMaterials;
        int materialID1;
        int materialID2;
        int[] materialID1_ToDelete = new int[1];
```

```

int[] materialID2_ToDelete = new int[1];
public void Main()
{
    modelName =
"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\dimxpert\bracket_auto_manual.sldprt";
swModel = swApp.OpenDoc6(modelName, (
int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
swModelDocExt = swModel.Extension;
// Get active configuration and create a new display
// state for this configuration
swConfig = (
Configuration)swModel.GetActiveConfiguration();
status = swConfig.CreateDisplayState(
"Display State 2");
swModel.ForceRebuild3(
true);
// Get active configuration and create another new
// display state for this configuration
swConfig = (
Configuration)swModel.GetActiveConfiguration();
status = swConfig.CreateDisplayState(
"Display State 3");
swModel.ForceRebuild3(
true);
// Create appearance
modelName =
"C:\Program Files\SolidWorks Corp\SolidWorks\data\graphics\materials\metal\steel\stainless steel treadplate.p2m";
swRenderMaterial = swModelDocExt.CreateRenderMaterial(materialName);
// Select a face and add the appearance to that face
status = swModelDocExt.SelectByID2(
"", "FACE", 0.07151920610502, 0.0952597996959, 0.009524999999996, false, 0, null, 0);
swSelMgr = (
SelectionMgr)swModel.SelectionManager;
swEntity = (
Entity)swSelMgr.GetSelectedObject6(1, -1);
status = swRenderMaterial.AddEntity(swEntity);
// Get the names of display states
displayStateNames = (
object[])swConfig.GetDisplayStates();
nbrDisplayStates = swConfig.GetDisplayStatesCount();
Debug.Print("This configuration's display states =");
for (i = 0; i <= (nbrDisplayStates - 1); i++)
{
    Debug.Print(" Display state name = " + displayStateNames[i]);
}
// Add appearance to all of the display states
status = swModelDocExt.AddDisplayStateSpecificRenderMaterial(swRenderMaterial, (
int)swDisplayStateOpts_e.swAllDisplayState, displayStateNames, out materialID1, out materialID2);
// Get the appearance IDs and names
swRenderMaterial.GetMaterialIds(
out materialID1, out materialID2);
Debug.Print(" Appearance IDs:");
Debug.Print(" ID1 = " + materialID1);
Debug.Print(" ID2 = " + materialID2);
nbrMaterials = swModelDocExt.GetRenderMaterialsCount2((
int)swDisplayStateOpts_e.swAllDisplayState, null);
Debug.Print(" Number of materials: " + nbrMaterials);
for (k = 0; k <= (nbrMaterials - 1); k++)
{
    Debug.Print(" Name of appearance " + (k + 1) + ": " + swModel.MaterialIdName);
}
double xcoord = 0;
double ycoord = 0;
double zcoord = 0;
swRenderMaterial.GetCenterPoint2(
out xcoord, out ycoord, out zcoord);
Debug.Print("");
Debug.Print("Texture-based appearance data:");
Debug.Print("X coordinate of center point: " + xcoord);
Debug.Print("Y coordinate of center point: " + ycoord);
Debug.Print("Z coordinate of center point: " + zcoord);
swRenderMaterial.GetUDirection2(

```

```

out xcoord, out ycoord, out zcoord);
Debug.Print("X coordinate of U direction: " + xcoord);
Debug.Print("Y coordinate of U direction: " + ycoord);
Debug.Print("Z coordinate of U direction: " + zcoord);
swRenderMaterial.GetVDirection2(
out xcoord, out ycoord, out zcoord);
Debug.Print("X coordinate of V direction: " + xcoord);
Debug.Print("Y coordinate of V direction: " + ycoord);
Debug.Print("Z coordinate of V direction: " + zcoord);
Debug.Print("");
swModel.ClearSelection2(
true);
swModel.ForceRebuild3(
true);
Debug.Print("Model has an appearance: " + swModelDocExt.HasMaterialPropertyValues());
object dispStates = null;
status = swRenderMaterial.SetLinkedDisplayStates((
int)swDisplayStateOpts_e.swAllDisplayState, displayStateNames);
dispStates = swRenderMaterial.GetLinkedDisplayStates();
object renderMaterials = null;
renderMaterials = swModelDocExt.GetRenderMaterials2((
int)swDisplayStateOpts_e.swAllDisplayState, null);
// Examine the display states of the active configuration
// to ensure that the specified appearance was applied to all
// display states
// Continue running the macro after your examination
System.Diagnostics.
Debugger.Break();
// Delete the appearance from the part
materialID1_ToDelete[0] = materialID1;
materialID2_ToDelete[0] = materialID2;
swModelDocExt.DeleteDisplayStateSpecificRenderMaterial((materialID1_ToDelete), (materialID2_ToDelete));
swModel.ForceRebuild3(
true);
// Examine the display states of the active configuration
// to ensure that the specified appearance was deleted from all
// display states
// Continue running the macro after your examination
System.Diagnostics.
Debugger.Break();
// Close the part without saving changes
modelName = swModel.GetTitle();
swApp.QuitDoc(modelName);
    }

    public SldWorks swApp;
}
}

```

Add and Delete Appearances from Specific Display States Example (VBA)

This example shows how to add an appearance to and delete an appearance from specific display states.

```

'-----
' Preconditions:
' 1. Specified model exists.
' 2. Specified appearance exists.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Creates Display State 2 and Display State 3 for the active
'    configuration.
' 2. Applies specified appearance to all display states of the active
'    configuration.
' 3. Press F5.
' 4. Deletes specified appearance from all display states of the active
'    configuration.
' 5. Press F5.
' 6. Closes document.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swConfig As SldWorks.Configuration
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swEntity As SldWorks.Entity
Dim swSelMgr As SldWorks.SelectionMgr
Dim swRenderMaterial As SldWorks.RenderMaterial
Dim displayStateNames As Variant
Dim status As Boolean
Dim modelName As String
Dim materialName As String
Dim errors As Long
Dim warnings As Long
Dim nbrDisplayStates As Long
Dim i As Long
Dim k As Long
Dim nbrMaterials As Long
Dim materialID1 As Long
Dim materialID2 As Long
Dim materialID1_ToDelete(0) As Long
Dim materialID2_ToDelete(0) As Long

Sub main()

    modelName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\dimxpert\bracket_auto_manual.sldprt"
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6(modelName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    ' Get active configuration and create a new display
    ' state for this configuration
    Set swConfig = swModel.GetActiveConfiguration
    status = swConfig.CreateDisplayState("Display State 2")

    swModel.ForceRebuild3 True

    ' Get active configuration and create another new
    ' display state for this configuration
    Set swConfig = swModel.GetActiveConfiguration
    status = swConfig.CreateDisplayState("Display State 3")

    swModel.ForceRebuild3 True

    ' Create appearance
    materialName = "C:\Program Files\SolidWorks Corp\SolidWorks\data\graphics\materials\metal\steel\stainless steel treadplate.p2m"
    Set swRenderMaterial = swModelDocExt.CreateRenderMaterial(materialName)

    ' Select a face and add the appearance to that face
    status = swModelDocExt.SelectByID("", "FACE", 0.07151920610502, 0.0952597996959, 0.009524999999996, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swEntity = swSelMgr.GetSelectedObject6(1, -1)
    status = swRenderMaterial.AddEntity(swEntity)

    ' Get the names of display states
    displayStateNames = swConfig.GetDisplayStates
    nbrDisplayStates = swConfig.GetDisplayStatesCount
    Debug.Print "This configuration's display states ="
    For i = 0 To (nbrDisplayStates - 1)
        Debug.Print "  Display state name = " & displayStateNames(i)
    Next i

    ' Add appearance to all of the display states
    status = swModelDocExt.AddDisplayStateSpecificRenderMaterial(swRenderMaterial, swAllDisplayState, displayStateNames,

```

```

materialID1, materialID2)

' Get the appearance IDs and names
swRenderMaterial.GetMaterialIds materialID1, materialID2
Debug.Print "    Appearance IDs:"
Debug.Print "        ID1 = " & materialID1
Debug.Print "        ID2 = " & materialID2
nbrMaterials = swModelDocExt.GetRenderMaterialsCount2(swAllDisplayState, Nothing)
Debug.Print "        Number of appearances: " & nbrMaterials
For k = 0 To (nbrMaterials - 1)
    Debug.Print "            Name of appearance " & (k + 1) & ": " & swModel.MaterialIdName
Next k

Dim xcoord As Double
Dim ycoord As Double
Dim zcoord As Double
swRenderMaterial.GetCenterPoint2 xcoord, ycoord, zcoord
Debug.Print ""
Debug.Print "Texture-based appearance data:"
Debug.Print "X coordinate of center point: " & xcoord
Debug.Print "Y coordinate of center point: " & ycoord
Debug.Print "Z coordinate of center point: " & zcoord

swRenderMaterial.GetUDirection2 xcoord, ycoord, zcoord
Debug.Print "X coordinate of U direction: " & xcoord
Debug.Print "Y coordinate of U direction: " & ycoord
Debug.Print "Z coordinate of U direction: " & zcoord

swRenderMaterial.GetVDirection2 xcoord, ycoord, zcoord
Debug.Print "X coordinate of V direction: " & xcoord
Debug.Print "Y coordinate of V direction: " & ycoord
Debug.Print "Z coordinate of V direction: " & zcoord
Debug.Print ""

swModel.ClearSelection2 True
swModel.ForceRebuild3 True

Debug.Print "Model has an appearance: " & swModelDocExt.HasMaterialPropertyValues

Dim dispStates As Variant
status = swRenderMaterial.SetLinkedDisplayStates(swAllDisplayState, displayStateNames)
dispStates = swRenderMaterial.GetLinkedDisplayStates

Dim renderMaterials As Variant
renderMaterials = swModelDocExt.GetRenderMaterials2(swAllDisplayState, Nothing)

' Examine the display states of the active configuration
' to ensure that the specified appearance was applied to all
' display states (click the ConfigurationManager tab and switch
' display states at bottom of the Configuration pane)
' Continue running the macro after your examination
Stop

' Delete the appearance from the part
materialID1_ToDelete(0) = materialID1
materialID2_ToDelete(0) = materialID2
swModelDocExt.DeleteDisplayStateSpecificRenderMaterial (materialID1_ToDelete), (materialID2_ToDelete)
swModel.ForceRebuild3 True

' Examine the display states of the active configuration
' to ensure that the specified appearance was deleted from all
' display states (click the ConfigurationManager tab and switch
' display states at bottom of the Configuration pane)
' Continue running the macro after your examination
Stop

' Close the part without saving changes
modelName = swModel.GetTitle
swApp.QuitDoc modelName

End Sub

```

Add and Delete Appearances from Specific Display States Example (VB.NET)

This example shows how to add an appearance to and delete an appearance from specific display states.

```
'-----
' Preconditions:
' 1. Specified model exists.
' 2. Specified appearance exists.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Creates Display State 2 and Display State 3 for the active
'    configuration.
' 2. Applies specified appearance to all display states of the active
'    configuration.
' 3. Press F5.
' 4. Deletes specified appearance from all display states of the active
'    configuration.
' 5. Press F5.
' 6. Closes document.
'-----
```

Imports

SolidWorks.Interop.sldworks

Imports

SolidWorks.Interop.swconst

Imports

System.Runtime.InteropServices

Imports

System

Imports

System.Diagnostics

Partial

Class SolidWorksMacro

Dim swModel As ModelDoc2

Dim swConfig As Configuration

Dim swModelDocExt As ModelDocExtension

Dim swEntity As Entity

Dim swSelMgr As SelectionMgr

Dim swRenderMaterial As RenderMaterial

Dim displayStateNames As Object

Dim status As Boolean

Dim modelName As String

Dim materialName As String

Dim errors As Integer

Dim warnings As Integer

Dim nbrDisplayStates As Integer

Dim i As Integer

Dim k As Integer

Dim nbrMaterials As Integer

Dim materialID1 As Integer

Dim materialID2 As Integer

Dim materialID1_ToDelete(0) As Integer

Dim materialID2_ToDelete(0) As Integer

Sub main()

 modelName =

 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\dimxpert\bracket_auto_manual.sldprt"

 swModel = swApp.OpenDoc6(modelName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,

 "", errors, warnings)

 swModelDocExt = swModel.Extension

 ' Get active configuration and create a new display

 ' state for this configuration

 swConfig = swModel.GetActiveConfiguration

 status = swConfig.CreateDisplayState(

```

"Display State 2")
swModel.ForceRebuild3(
True)
' Get active configuration and create another new
' display state for this configuration
swConfig = swModel.GetActiveConfiguration
status = swConfig.CreateDisplayState(
"Display State 3")
swModel.ForceRebuild3(
True)
' Create appearance
materialName =
"C:\Program Files\SolidWorks Corp\SolidWorks\data\graphics\materials\metal\steel\stainless steel treadplate.p2m"
swRenderMaterial = swModelDocExt.CreateRenderMaterial(materialName)
' Select a face and add the appearance to that face
status = swModelDocExt.SelectByID(
"", "FACE", 0.07151920610502, 0.0952597996959, 0.009524999999996, False, 0, Nothing, 0)
swSelMgr = swModel.SelectionManager
swEntity = swSelMgr.GetSelectedObject6(1, -1)
status = swRenderMaterial.AddEntity(swEntity)
' Get the names of display states
displayStateNames = swConfig.GetDisplayStates
nbrDisplayStates = swConfig.GetDisplayStatesCount
Debug.Print(
"This configuration's display states =")
For i = 0 To (nbrDisplayStates - 1)
Debug.Print(
" Display state name = " & displayStateNames(i))
Next i
' Add appearance to all of the display states
status = swModelDocExt.AddDisplayStateSpecificRenderMaterial(swRenderMaterial, swDisplayStateOpts_e.swAllDisplayState,
displayStateNames, materialID1, materialID2)
' Get the appearance IDs and names
swRenderMaterial.GetMaterialIds(materialID1, materialID2)
Debug.Print(
" Appearance IDs:")
Debug.Print(
" ID1 = " & materialID1)
Debug.Print(
" ID2 = " & materialID2)
nbrMaterials = swModelDocExt.GetRenderMaterialsCount2(swDisplayStateOpts_e.swAllDisplayState,
Nothing)
Debug.Print(
" Number of appearances: " & nbrMaterials)
For k = 0 To (nbrMaterials - 1)
Debug.Print(
" Name of appearance " & (k + 1) & ": " & swModel.MaterialIdName)
Next k
Dim xcoord As Double
Dim ycoord As Double
Dim zcoord As Double
swRenderMaterial.GetCenterPoint2(xcoord, ycoord, zcoord)
Debug.Print(
"")
Debug.Print(
"Texture-based appearance data:")
Debug.Print(
"X coordinate of center point: " & xcoord)
Debug.Print(
"Y coordinate of center point: " & ycoord)
Debug.Print(
"Z coordinate of center point: " & zcoord)
swRenderMaterial.GetUDirection2(xcoord, ycoord, zcoord)
Debug.Print(
"X coordinate of U direction: " & xcoord)
Debug.Print(
"Y coordinate of U direction: " & ycoord)
Debug.Print(
"Z coordinate of U direction: " & zcoord)
swRenderMaterial.GetVDirection2(xcoord, ycoord, zcoord)
Debug.Print(

```

```

"X coordinate of V direction: " & xcoord)
Debug.Print(

"Y coordinate of V direction: " & ycoord)
Debug.Print(

"Z coordinate of V direction: " & zcoord)
Debug.Print(

"" )
swModel.ClearSelection2(

True)
swModel.ForceRebuild3(

True)
Debug.Print(

"Model has an appearance: " & swModelDocExt.HasMaterialPropertyValues)

Dim dispStates As Object
status = swRenderMaterial.SetLinkedDisplayStates(swDisplayStateOpts_e.swAllDisplayState, displayStateNames)
dispStates = swRenderMaterial.GetLinkedDisplayStates

Dim renderMaterials As Object
renderMaterials = swModelDocExt.GetRenderMaterials2(swDisplayStateOpts_e.swAllDisplayState,

Nothing)

' Examine the display states of the active configuration
' to ensure that the specified appearance was applied to all
' display states
' Continue running the macro after your examination
Stop

' Delete the appearance from the part
materialID1_ToDelete(0) = materialID1
materialID2_ToDelete(0) = materialID2
swModelDocExt.DeleteDisplayStateSpecificRenderMaterial((materialID1_ToDelete), (materialID2_ToDelete))
swModel.ForceRebuild3(

True)

' Examine the display states of the active configuration
' to ensure that the specified appearance was deleted from all
' display states
' Continue running the macro after your examination
Stop

' Close the part without saving changes
modelName = swModel.GetTitle
swApp.QuitDoc(modelName)

End Sub

Public swApp As SldWorks

End
Class

```

Add and Edit Distance Mate Example (C#)

This example shows how to add and edit a cylindrical distance mate.

```
//-----
// Preconditions:
// 1. Ensure the specified template exists.
// 2. Open public_documents\samples\tutorial\api\cylinder20.sldprt.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Creates and saves a new cylindrical part.
// 2. Adds two cylindrical entities to a new assembly.
// 3. Creates a distance mate between the cylindrical entities.
// 4. Edits the distance mate to change the distance from 0.2 to 0.3.
// 5. Inspect the Immediate window, the graphics area, and the Mates folder
//    of the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using System.Diagnostics;

namespace AddDistanceMate_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            AssemblyDoc swAssembly = default(AssemblyDoc);
            ModelDoc2 Part = default(ModelDoc2);
            string AssemblyTitle = null;
            Component2 swInsertedComponent = default(Component2);
            Feature swFeat = default(Feature);
            bool boolstatus = false;
            int longstatus = 0;
            int longwarnings = 0;
            double swSheetWidth = 0;
            double swSheetHeight = 0;
            Mate2 swMate = default(Mate2);
            ModelDoc2 tmpObj = default(ModelDoc2);
            int errors = 0;
            Entity swEnt1 = default(Entity);
            Entity swEnt2 = default(Entity);

            Part = (ModelDoc2)swApp.ActiveDoc;

            // Shell the active part
            boolstatus = Part.Extension.SelectByRay(-0.0108900020093756, 0.0655319999998483, -0.00515652172191494, -0.40003602677931
            0);
            Part.InsertFeatureShell(0.00254, false);

            // Save the shelled part
            longstatus = Part.SaveAs3("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20_
            sldprt");

            // Create a new assembly
            swSheetWidth = 0;
            swSheetHeight = 0;
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2018\\templates\\Assembly.asmdot", 0, swShe

            // Insert a cylinder20_shell component
            AssemblyTitle = Part.GetTitle();
            tmpObj = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20_s
            ldprt");
            swAssembly = (AssemblyDoc)swApp.ActivateDoc3(AssemblyTitle, true, 0, longstatus);
            swInsertedComponent = swAssembly.AddComponent5("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tut
            cylinder20_shell.sldprt");
            swApp.CloseDoc("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20_shell.sldp

            // Insert another cylinder20_shell component
            tmpObj = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20_s
            ldprt");
            swAssembly = (AssemblyDoc)swApp.ActivateDoc3(AssemblyTitle, true, 0, errors);
            swInsertedComponent = swAssembly.AddComponent5("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tut
            cylinder20_shell.sldprt");
            swApp.CloseDoc("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20_shell.sldp

            // Select two cylindrical entities
            boolstatus = Part.Extension.SelectByRay(-0.140174514310559, 0.00237221117538411, 0.0264513806530431, -0.400036026779312,
            0);
            swEnt1 = (Entity)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            boolstatus = Part.Extension.SelectByRay(0.0679787981690652, -0.00725673614920197, -0.0758574895979791, -0.40003602677931
            0);
            swEnt2 = (Entity)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);

            swEnt1.Select4(true, null);
            swEnt2.Select4(true, null);

            // Add a center-to-center distance mate between the selected cylindrical entities
```

```

swMate = swAssembly.AddDistanceMate(2, false, 0.2, 0, 0, 1, 1, out errors);
Debug.Print("First arc condition as defined in swDistanceMateArcConditions_e: " + swMate.DistanceFirstArcCondition);
Debug.Print("Second arc condition as defined in swDistanceMateArcConditions_e: " + swMate.DistanceSecondArcCondition);
swFeat = (Feature)swMate;

Part.EditRebuild3();

// Edit the distance mate to change the distance from 0.2 to 0.3
boolstatus = Part.Extension.SelectByRay(-0.0936626010895907, 0.000678476678046991, -0.000454698905400619, -0.40003602671
0);
swEnt1 = (Entity)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
boolstatus = Part.Extension.SelectByRay(0.0803986691953469, -0.00107796570199525, -0.0914337018962783, -0.40003602677931
0);
swEnt2 = (Entity)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);

swEnt1.Select4(true, null);
swEnt2.Select4(true, null);
swFeat.Select2(true, 0);

swAssembly.EditDistanceMate(2, false, 0.3, 0, 0, 1, 1, out errors);

Part.EditRebuild3();
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;
}
}

```

Add and Edit Distance Mate Example (VBA)

This example shows how to add and edit a cylindrical distance mate.

```

'-----
' Preconditions:
' 1. Ensure the specified template exists.
' 2. Open public_documents\samples\tutorial\api\cylinder20.sldprt.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Creates and saves a new cylindrical part.
' 2. Adds two cylindrical entities to a new assembly.
' 3. Creates a distance mate between the cylindrical entities.
' 4. Edits the distance mate to change the distance from 0.2 to 0.3.
' 5. Inspect the Immediate window, the graphics area, and the Mates folder
'    of the FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim swAssembly As SldWorks.AssemblyDoc
Dim Part As SldWorks.ModelDoc2
Dim AssemblyTitle As String
Dim swInsertedComponent As Component2
Dim swFeat As Feature
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long
Dim swSheetWidth As Double
Dim swSheetHeight As Double
Dim swMate As Mate2
Dim tmpObj As ModelDoc2
Dim errors As Long
Dim swEnt1 As SldWorks.Entity
Dim swEnt2 As SldWorks.Entity
Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    ' Shell the active part
    boolstatus = Part.Extension.SelectByRay(-1.08900020093756E-02, 6.55319999998483E-02, -5.15652172191494E-03, -0.400036026779312, -0.515038074910024, -0.758094294050284, 1.67637314537445E-03, 2, False, 1, 0)
    Part.InsertFeatureShell 0.00254, False

    ' Save the shelled part
    longstatus = Part.SaveAs3("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt", 0, 2)

    ' Create a new assembly
    swSheetWidth = 0
    swSheetHeight = 0
    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2018\templates\Assembly.asmdot", 0, swSheetWidth, swSheetHeight)

    ' Insert a cylinder20_shell component
    AssemblyTitle = Part.GetTitle
    Set tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt", 1, 32, "", longstatus, longwarnings)
    Set Part = swApp.ActivateDoc3(AssemblyTitle, True, 0, longstatus)
    Set swInsertedComponent = Part.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt", 0, "", False, "", 0.119562469422817, -1.02308109635487E-02, -4.74663286004215E-02)
    swApp.CloseDoc "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt"

    ' Insert another cylinder20_shell component
    Set tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt", 1, 32, "", errors, longwarnings)
    Set Part = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)
    Set swInsertedComponent = Part.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt", 0, "", False, "", -0.130620346986689, -1.01738580269739E-02, 0.084551733918488)
    swApp.CloseDoc "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt"

    ' Select two cylindrical entities
    boolstatus = Part.Extension.SelectByRay(-0.140174514310559, 2.37221117538411E-03, 2.64513806530431E-02, -0.400036026779312, -0.515038074910024, -0.758094294050284, 8.6563679245283E-04, 2, False, 1, 0)
    Set swEnt1 = Part.SelectionManager.GetSelectedObject6(1, -1)
    boolstatus = Part.Extension.SelectByRay(6.79787981690652E-02, -7.25673614920197E-03, -7.58574895979791E-02, -0.400036026779312, -0.515038074910024, -0.758094294050284, 6.36203082166533E-04, 2, True, 1, 0)
    Set swEnt2 = Part.SelectionManager.GetSelectedObject6(1, -1)

    swEnt1.Select4 True, Nothing
    swEnt2.Select4 True, Nothing

```

```

' Add a center-to-center distance mate between the selected cylindrical entities
Set swAssembly = Part

Set swMate = swAssembly.AddDistanceMate(2, False, 0.2, 0, 0, 1, 1, errors)
Debug.Print "First arc condition as defined in swDistanceMateArcConditions_e: " & swMate.DistanceFirstArcCondition
Debug.Print "Second arc condition as defined in swDistanceMateArcConditions_e: " & swMate.DistanceSecondArcCondition
Set swFeat = swMate

Part.EditRebuild3

' Edit the distance mate to change the distance from 0.2 to 0.3
boolstatus = Part.Extension.SelectByRay(-9.36626010895907E-02, 6.78476678046991E-04, -4.54698905400619E-04, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 8.08436123348018E-04, 2, True, 1, 0)
Set swEnt1 = Part.SelectionManager.GetSelectedObject6(1, -1)
boolstatus = Part.Extension.SelectByRay(8.03986691953469E-02, -1.07796570199525E-03, -9.14337018962783E-02, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 8.08436123348018E-04, 2, True, 2, 0)
Set swEnt2 = Part.SelectionManager.GetSelectedObject6(2, -1)

swEnt1.Select4 True, Nothing
swEnt2.Select4 True, Nothing
swFeat.Select2 True, 0

swAssembly.EditDistanceMate 2, False, 0.3, 0, 0, 1, 1, errors

Part.EditRebuild3
End Sub

```

Add and Edit Distance Mate Example (VB.NET)

This example shows how to add and edit a cylindrical distance mate.

```

'-----
' Preconditions:
' 1. Ensure the specified template exists.
' 2. Open public_documents\samples\tutorial\api\cylinder20.sldprt.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Creates and saves a new cylindrical part.
' 2. Adds two cylindrical entities to a new assembly.
' 3. Creates a distance mate between the cylindrical entities.
' 4. Edits the distance mate to change the distance from 0.2 to 0.3.
' 5. Inspect the Immediate window, the graphics area, and the Mates folder
'    of the FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Imports SolidWorks.Interop.sldworks

Partial Class SolidWorksMacro

Public Sub main()

```

    Dim swAssembly As AssemblyDoc
    Dim Part As ModelDoc2
    Dim AssemblyTitle As String
    Dim swInsertedComponent As Component2
    Dim swFeat As Feature
    Dim boolstatus As Boolean
    Dim longstatus As Integer, longwarnings As Integer
    Dim swSheetWidth As Double
    Dim swSheetHeight As Double
    Dim swMate As Mate2
    Dim tmpObj As ModelDoc2
    Dim errors As Integer
    Dim swEnt1 As Entity
    Dim swEnt2 As Entity

```

Part = swApp.ActiveDoc

```

' Shell the active part
boolstatus = Part.Extension.SelectByRay(-0.0108900020093756, 0.0655319999998483, -0.00515652172191494, -0.400036026779312, -
Part.InsertFeatureShell(0.00254, False)

```

```

' Save the shelled part
longstatus = Part.SaveAs3("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt

```

```

' Create a new assembly
swSheetWidth = 0
swSheetHeight = 0
Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2018\templates\Assembly.asmdot", 0, swSheetWidth, swSheetHeig

```

```

' Insert a cylinder20_shell component
AssemblyTitle = Part.GetTitle
tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt",
Part = swApp.ActivateDoc3(AssemblyTitle, True, 0, longstatus)
swInsertedComponent = Part.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder
swApp.CloseDoc("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt")

```

```

' Insert another cylinder20_shell component
tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt",
Part = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)
swInsertedComponent = Part.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder
swApp.CloseDoc("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20_shell.sldprt")

```

```

' Select two cylindrical entities
boolstatus = Part.Extension.SelectByRay(-0.140174514310559, 0.00237221117538411, 0.0264513806530431, -0.400036026779312, -0.
swEnt1 = Part.SelectionManager.GetSelectedObject6(1, -1)
boolstatus = Part.Extension.SelectByRay(0.0679787981690652, -0.00725673614920197, -0.0758574895979791, -0.400036026779312, -
swEnt2 = Part.SelectionManager.GetSelectedObject6(1, -1)

```

```

swEnt1.Select4(True, Nothing)
swEnt2.Select4(True, Nothing)

```

```

' Add a center-to-center distance mate between the selected cylindrical entities
swAssembly = Part
swMate = swAssembly.AddDistanceMate(2, False, 0.2, 0, 0, 1, 1, errors)
Debug.Print("First arc condition as defined in swDistanceMateArcConditions_e: " & swMate.DistanceFirstArcCondition)
Debug.Print("Second arc condition as defined in swDistanceMateArcConditions_e: " & swMate.DistanceSecondArcCondition)
swFeat = swMate

```

Part.EditRebuild3

```

' Edit the distance mate to change the distance from 0.2 to 0.3

```

```
boolstatus = Part.Extension.SelectByRay(-0.0936626010895907, 0.000678476678046991, -0.000454698905400619, -0.400036026779312  
swEnt1 = Part.SelectionManager.GetSelectedObject6(1, -1)  
boolstatus = Part.Extension.SelectByRay(0.0803986691953469, -0.00107796570199525, -0.0914337018962783, -0.400036026779312, -  
swEnt2 = Part.SelectionManager.GetSelectedObject6(2, -1)
```

```
swEnt1.Select4(True, Nothing)  
swEnt2.Select4(True, Nothing)  
swFeat.Select2(True, 0)
```

```
swAssembly.EditDistanceMate(2, False, 0.3, 0, 0, 1, 1, errors)
```

```
Part.EditRebuild3
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Add and Edit Misaligned Symmetric Concentric Mate Example (C#)

This example shows how to add and edit a misaligned symmetric concentric mate between components.

```
//-----
// Preconditions:
// 1. Verify that the parts to open and assembly template exist.
// 2. Open a new session of SOLIDWORKS Design to ensure that the assembly
//    document's title is Assem1.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Opens wheel_hub.sldprt.
// 2. Opens a new assembly document titled Assem1 and adds
//    wheel_hub.sldprt as a component.
// 3. Opens and adds beam_with_holes.sldprt as another component.
// 4. Adds a concentric mate between the components.
// 5. Adds a misaligned symmetric concentric mate
//    between the components.
// 6. Edits the misaligned symmetric concentric mate.
// 7. Examine the Immediate window, graphics area, and the mates
//    in the Mates folder in the FeatureManager design tree.
//
// NOTE: Because the parts are used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            Component2 swInsertedComponent = default(Component2);
            MathUtility swMathUtil = default(MathUtility);
            MathTransform swTransform = default(MathTransform);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Mate2 swMate = default(Mate2);
            Mate2 swLinkedMate = default(Mate2);
            Component2 swComp = default(Component2);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            Feature swFeature = default(Feature);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            double swSheetWidth = 0;
            double swSheetHeight = 0;
            string AssemblyTitle = null;
            double[] TransformData = new double[16];
            object TransformDataObj = null;

            //Open the part
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\wheel_hub.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swApp.ActivateDoc2("wheel_hub.sldprt", false, ref errors);

            //Create an assembly
            swSheetWidth = 0;
            swSheetHeight = 0;
            swAssembly = (AssemblyDoc)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2018\\templates\\Assembly.asmdot", 0, swSheetWidth, swShe
etHeight);
            swModel = (ModelDoc2)swApp.ActivateDoc3(AssemblyTitle, true, 0, ref errors);
            AssemblyTitle = swModel.GetTitle();

            //Add the part to the assembly
            swInsertedComponent = (Component2)swAssembly.AddComponent5(fileName, (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSel
ectedConfig, "", false, "", -0.000181145833835217, 0.000107469465717713, 2.25750183631135E-05);
            swApp.CloseDoc(fileName);
            TransformData[0] = 1;
            TransformData[1] = 0;
            TransformData[2] = 0;
            TransformData[3] = 0;
            TransformData[4] = 1;
            TransformData[5] = 0;
            TransformData[6] = 0;
            TransformData[7] = 0;
            TransformData[8] = 1;
            TransformData[9] = 0;
            TransformData[10] = 0;
            TransformData[11] = 0;
            TransformData[12] = 1;
            TransformData[13] = 0;
            TransformData[14] = 0;
            TransformData[15] = 0;
            TransformDataObj = (object)TransformData;
            swMathUtil = (MathUtility)swApp.GetMathUtility();
            swTransform = (MathTransform)swMathUtil.CreateTransform((TransformDataObj));
            status = swInsertedComponent.SetTransformAndSolve2(swTransform);

            //Open and add another part to the assembly
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\beam_with_holes.sldprt";
            swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_AutoMissingConfig, "", ref errors, ref war
nings);
            swModel = (ModelDoc2)swApp.ActivateDoc3(AssemblyTitle, true, 0, ref errors);
            swAssembly = (AssemblyDoc)swModel;
            swInsertedComponent = (Component2)swAssembly.AddComponent5(fileName, (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSel
ectedConfig, "", false, "", -0.0543800538871437, -0.10948954091873, 0.0944189997389913);
            swApp.CloseDoc(fileName);
            TransformData[0] = 1;
            TransformData[1] = 0;
            TransformData[2] = 0;
```

```

TransformData[3] = 0;
TransformData[4] = 1;
TransformData[5] = 0;
TransformData[6] = 0;
TransformData[7] = 0;
TransformData[8] = 1;
TransformData[9] = -0.179380053887144;
TransformData[10] = -0.0894895409187302;
TransformData[11] = 0.0744189997389913;
TransformData[12] = 1;
TransformData[13] = 0;
TransformData[14] = 0;
TransformData[15] = 0;
TransformDataObj = (object)TransformData;
swMathUtil = (MathUtility)swApp.GetMathUtility();
swTransform = (MathTransform)swMathUtil.CreateTransform((TransformDataObj));

//Add a concentric mate
swModelDocExt = (ModelDocExtension)swModel.Extension;
status = swModelDocExt.SelectByRay(-0.158811046822791, -0.11180606883903, 0.110029416510201, -0.400036026779312, -0.515038074910024, -0.758094
294050284, 0.000877710636365774, 2, true, 1,
0);
status = swModelDocExt.SelectByRay(-0.0595820179927387, -0.0282041473137156, 0.0120989536100637, -0.400036026779312, -0.515038074910024, -0.75
8094294050284, 0.000877710636365774, 2, true, 1,
0);
swMate = (Mate2)swAssembly.AddMate5((int)swMateType_e.swMateCONCENTRIC, (int)swMateAlign_e.swMateAlignALIGNED, false, 0.128546596088087, 0.025
4, 0.0254, 0.0254, 0.0254, 0, 0.5235987755983,
0.5235987755983, false, false, 0, out errors);
swModel.ClearSelection2(true);
swModel.EditRebuild3();

//Move a component in order to add another mate
status = swModelDocExt.SelectByID("beam with holes-1@Assem1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
TransformData[0] = 0.999268594246539;
TransformData[1] = -0.0382397247969312;
TransformData[2] = 0;
TransformData[3] = 0.0382397247969312;
TransformData[4] = 0.999268594246539;
TransformData[5] = 0;
TransformData[6] = 0;
TransformData[7] = 0;
TransformData[8] = 1;
TransformData[9] = -0.0817169203602251;
TransformData[10] = -0.00405863499514564;
TransformData[11] = 0.158954897029145;
TransformData[12] = 1;
TransformData[13] = 0;
TransformData[14] = 0;
TransformData[15] = 0;
TransformDataObj = (object)TransformData;
swMathUtil = (MathUtility)swApp.GetMathUtility();
swTransform = (MathTransform)swMathUtil.CreateTransform((TransformDataObj));
swSelectionManager = (SelectionMgr)swModel.SelectionManager;
swComp = (Component2)swSelectionManager.GetSelectedObjectsComponent4(1, -1);
status = swComp.SetTransformAndSolve2(swTransform);

//Add a misaligned symmetric concentric mate
status = swModelDocExt.SelectByRay(-0.0362917984178921, -0.0292363280984205, 0.194521666713399, -0.400036026779312, -0.515038074910024, -0.758
094294050284, 0.000877710636365774, 2, true, 1, 0);
status = swModelDocExt.SelectByRay(0.054967212945769, -0.0278611795938559, 0.0112380964281442, -0.400036026779312, -0.515038074910024, -0.7580
94294050284, 0.000877710636365774, 2, true, 1, 0);
Debug.Print("Misaligned concentric mate:");
swMate = (Mate2)swAssembly.AddConcentricMateWithTolerance((int)swMateAlign_e.swMateAlignALIGNED, (int)swConcentricAlignmentType_e.swConcentric
AlignSymmetric, false, 0, out errors);
if (errors == 1)
{
    Debug.Print(" Added");
}
else
{
    Debug.Print(" Not added");
}

swModel.ClearSelection2(true);

//Edit the misaligned symmetric concentric mate
//and change its position from symmetric to aligned
status = swModelDocExt.SelectByRay(0.0548392523382404, -0.0277425865184, 0.00906375356890976, -0.400036026779312, -0.515038074910024, -0.75809
4294050284, 0.00107347349835279, 2, false, 0, 0);
status = swModelDocExt.SelectByRay(0.00980255664001106, -0.0292099642897483, 0.193646683964346, -0.400036026779312, -0.515038074910024, -0.758
094294050284, 0.00107347349835279, 2, true, 0, 0);
status = swModelDocExt.SelectByID("Concentric2", "MATE", 0, 0, 0, true, 0, null, 0);
swAssembly.EditConcentricMate((int)swMateAlign_e.swMateAlignALIGNED, (int)swConcentricAlignmentType_e.swConcentricAlignThisMate, true, 0.095,
out errors);

swModel.ClearSelection2(true);

if (errors == 1)
{
    Debug.Print(" Edited");
    status = swModelDocExt.SelectByID2("Concentric2", "MATE", 0, 0, 0, true, 0, null, 0);
    swFeature = (Feature)swSelectionManager.GetSelectedObject6(1, -1);
    swMate = (Mate2)swFeature.GetSpecificFeature2();

    //Get the linked concentric mate
    swLinkedMate = (Mate2)swMate.GetLinkedMate();
    //Get current misaligned deviation
    Debug.Print(" Current misalignment deviation: " + swLinkedMate.GetCurrentMisalignedDeviation());
    //Do not use document property value for misalignment deviation
    swMate.SetUseMisalignedDeviationDocumentProperty(false);
    Debug.Print(" Use document property? " + swMate.GetUseMisalignedDeviationDocumentProperty());
    //Change mate alignment type to symmetric
    swLinkedMate.SetConcentricAlignmentType((int)swConcentricAlignmentType_e.swConcentricAlignSymmetric);
    Debug.Print(" Concentric alignment type as defined in swConcentricAlignmentType_e: " + swLinkedMate.GetConcentric
    //Set maximum misalignment deviation
    swMate.SetMaximumMisalignedDeviation(0.102);
    Debug.Print(" Maximum misalignment deviation: " + swMate.GetMaximumMisalignedDeviation());
}
else
{
    Debug.Print(" Not edited");
}

```

```
        swModel.ClearSelection2(true);
        swModel.EditRebuild3();
    }

    /// <summary>
    ///   The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Add and Edit Misaligned Symmetric Concentric Mate Example (VBA)

This example shows how to add and edit a misaligned symmetric concentric mate between components.

```

'-----
' Preconditions:
' 1. Verify that the parts to open and assembly template exist.
' 2. Open a new session of SOLIDWORKS Design to ensure that the assembly
'    document's title is Assem1.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens wheel_hub.sldprt.
' 2. Opens a new assembly document titled Assem1 and adds
'    wheel_hub.sldprt as a component.
' 3. Opens and adds beam with holes.sldprt as another component.
' 4. Adds a concentric mate between the components.
' 5. Adds a misaligned symmetric concentric mate
'    between the components.
' 6. Edits the misaligned symmetric concentric mate.
' 7. Examine the Immediate window, graphics area, and the mates
'    in the Mates folder in the FeatureManager design tree.
'
' NOTE: Because the parts are used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim swInsertedComponent As SldWorks.Component2
Dim swMathUtil As SldWorks.MathUtility
Dim swTransform As SldWorks.MathTransform
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMate As SldWorks.Mate2
Dim swLinkedMate As SldWorks.Mate2
Dim swComp As SldWorks.Component2
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim swSheetWidth As Double
Dim swSheetHeight As Double
Dim AssemblyTitle As String
Dim TransformData() As Double
Dim TransformDataVariant As Variant

Sub main()

    Set swApp = Application.SldWorks

    'Open the part
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wheel_hub.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    swApp.ActivateDoc2 "wheel_hub.sldprt", False, errors

    'Create an assembly
    swSheetWidth = 0
    swSheetHeight = 0
    Set swAssembly = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2018\templates\Assembly.asmdot", 0, swSheetWidth, swSheetHeight)
    AssemblyTitle = swAssembly.GetTitle

    'Add the part to the assembly
    Set swInsertedComponent = swAssembly.AddComponent5(fileName, swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "",
    False, "", -1.81145833835217E-04, 1.07469465717713E-04, 2.25750183631135E-05)
    swApp.CloseDoc fileName
    ReDim TransformData(0 To 15) As Double
    TransformData(0) = 1
    TransformData(1) = 0
    TransformData(2) = 0
    TransformData(3) = 0
    TransformData(4) = 1
    TransformData(5) = 0
    TransformData(6) = 0
    TransformData(7) = 0
    TransformData(8) = 1
    TransformData(9) = 0
    TransformData(10) = 0
    TransformData(11) = 0
    TransformData(12) = 1
    TransformData(13) = 0
    TransformData(14) = 0
    TransformData(15) = 0
    TransformDataVariant = TransformData
    Set swMathUtil = swApp.GetMathUtility()
    Set swTransform = swMathUtil.CreateTransform((TransformDataVariant))
    status = swInsertedComponent.SetTransformAndSolve2(swTransform)

    'Add another part to the assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\beam with holes.sldprt"
    swApp.OpenDoc6 fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_AutoMissingConfig, "", errors, warnings
    Set swModel = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)
    Set swAssembly = swModel
    Set swInsertedComponent = swAssembly.AddComponent5(fileName, swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "",

```

```

False, "", -5.43800538871437E-02, -0.10948954091873, 9.44189997389913E-02)
swApp.CloseDoc fileName
ReDim TransformData(0 To 15) As Double
TransformData(0) = 1
TransformData(1) = 0
TransformData(2) = 0
TransformData(3) = 0
TransformData(4) = 1
TransformData(5) = 0
TransformData(6) = 0
TransformData(7) = 0
TransformData(8) = 1
TransformData(9) = -0.179380053887144
TransformData(10) = -8.94895409187302E-02
TransformData(11) = 7.44189997389913E-02
TransformData(12) = 1
TransformData(13) = 0
TransformData(14) = 0
TransformData(15) = 0
TransformDataVariant = TransformData
Set swMathUtil = swApp.GetMathUtility()
Set swTransform = swMathUtil.CreateTransform((TransformDataVariant))

'Add a concentric mate
Set swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByRay(-0.158811046822791, -0.11180606883903, 0.110029416510201, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 8.77710636365774E-04, 2, True, 1, 0)
status = swModelDocExt.SelectByRay(-5.95820179927387E-02, -2.82041473137156E-02, 1.20989536100637E-02, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 8.77710636365774E-04, 2, True, 1, 0)
Set swMate = swAssembly.AddMate5(swMateType_e.swMateCONCENTRIC, swMateAlign_e.swMateAlignALIGNED, False, 0.128546596088087, 0.0254, 0.0254, 0.0254,
0.0254, 0, 0.5235987755983, 0.5235987755983, False, False, 0, errors)
swModel.ClearSelection2 True
swModel.EditRebuild3

'Move a component in order to add another mate
status = swModelDocExt.SelectByID2("beam with holes-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
ReDim TransformData(0 To 15) As Double
TransformData(0) = 0.999268594246539
TransformData(1) = -3.82397247969312E-02
TransformData(2) = 0
TransformData(3) = 3.82397247969312E-02
TransformData(4) = 0.999268594246539
TransformData(5) = 0
TransformData(6) = 0
TransformData(7) = 0
TransformData(8) = 1
TransformData(9) = -8.17169203602251E-02
TransformData(10) = -4.05863499514564E-03
TransformData(11) = 0.158954897029145
TransformData(12) = 1
TransformData(13) = 0
TransformData(14) = 0
TransformData(15) = 0
TransformDataVariant = TransformData
Set swMathUtil = swApp.GetMathUtility()
Set swTransform = swMathUtil.CreateTransform((TransformDataVariant))
Set swSelectionManager = swModel.SelectionManager
Set swComp = swSelectionManager.GetSelectedObjectsComponent4(1, -1)
status = swComp.SetTransformAndSolve2(swTransform)

'Add a misaligned symmetric concentric mate
status = swModelDocExt.SelectByRay(-3.62917984178921E-02, -2.92363280984205E-02, 0.194521666713399, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 8.77710636365774E-04, 2, True, 1, 0)
status = swModelDocExt.SelectByRay(0.054967212945769, -2.78611795938559E-02, 1.12380964281442E-02, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 8.77710636365774E-04, 2, True, 1, 0)
Debug.Print "Misaligned concentric mate:"
Set swMate = swAssembly.AddConcentricMateWithTolerance(swMateAlign_e.swMateAlignALIGNED, swConcentricAlignmentType_e.swConcentricAlignSymmetric,
False, 0, errors)
If errors = 1 Then
    Debug.Print " Added"
Else
    Debug.Print " Not added"
End If

swModel.ClearSelection2 True

'Edit the misaligned symmetric concentric mate
'and change its position from symmetric to aligned
status = swModelDocExt.SelectByRay(5.48392523382404E-02, -0.0277425865184, 9.06375356890976E-03, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 1.07347349835279E-03, 2, False, 0, 0)
status = swModelDocExt.SelectByRay(9.80255664001106E-03, -2.92099642897483E-02, 0.193646683964346, -0.400036026779312, -0.515038074910024,
-0.758094294050284, 1.07347349835279E-03, 2, True, 0, 0)
status = swModelDocExt.SelectByID2("Concentric2", "MATE", 0, 0, 0, True, 0, Nothing, 0)
swAssembly.EditConcentricMate swMateAlign_e.swMateAlignALIGNED, swConcentricAlignmentType_e.swConcentricAlignThisMate, True, 0.095, errors

swModel.ClearSelection2 True

If errors = 1 Then
    Debug.Print " Edited"
    status = swModelDocExt.SelectByID2("Concentric2", "MATE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeature = swSelectionManager.GetSelectedObject6(1, -1)
    Set swMate = swFeature.GetSpecificFeature2

    'Get the linked concentric mate
    Set swLinkedMate = swMate.GetLinkedMate

    'Get current misaligned deviation
    Debug.Print " Current misalignment deviation: " & swLinkedMate.GetCurrentMisalignedDeviation

    'Do not use document property value for misalignment deviation
    swMate.SetUseMisalignedDeviationDocumentProperty False
    Debug.Print " Use document property? " & swMate.GetUseMisalignedDeviationDocumentProperty

```

```
'Change mate alignment type to symmetric
swLinkedMate.SetConcentricAlignmentType (swConcentricAlignmentType_e.swConcentricAlignSymmetric)
Debug.Print "    Concentric alignment type as defined in swConcentricAlignmentType_e: " &
swLinkedMate.GetConcentricAlignmentType

'Set maximum misalignment deviation
swMate.SetMaximumMisalignedDeviation 0.102
Debug.Print "    Maximum misalignment deviation: " & swMate.GetMaximumMisalignedDeviation
Else
    Debug.Print "    Not edited"
End If

swModel.ClearSelection2 True
swModel.EditRebuild3

End Sub
```

Add and Edit Misaligned Symmetric Concentric Mate Example (VB.NET)

This example shows how to add and edit a misaligned symmetric concentric mate between components.

```

'-----
' Preconditions:
' 1. Verify that the parts to open and assembly template exist.
' 2. Open a new session of SOLIDWORKS Design to ensure that the assembly
'    document's title is Assem1.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens wheel_hub.sldprt.
' 2. Opens a new assembly document titled Assem1 and adds
'    wheel_hub.sldprt as a component.
' 3. Opens and adds beam with holes.sldprt as another component.
' 4. Adds a concentric mate between the components.
' 5. Adds a misaligned symmetric concentric mate
'    between the components.
' 6. Edits the misaligned symmetric concentric mate.
' 7. Examine the Immediate window, graphics area, and the mates
'    in the Mates folder in the FeatureManager design tree.
'
' NOTE: Because the parts are used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swAssembly As AssemblyDoc
        Dim swInsertedComponent As Component2
        Dim swMathUtil As MathUtility
        Dim swTransform As MathTransform
        Dim swModelDocExt As ModelDocExtension
        Dim swMate As Mate2
Dim swLinkedMate As Mate2
        Dim swComp As Component2
        Dim swSelectionManager As SelectionMgr
        Dim swFeature As Feature
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim swSheetWidth As Double
        Dim swSheetHeight As Double
        Dim AssemblyTitle As String
        Dim TransformData(15) As Double
        Dim TransformDataObj As Object

'Open the part
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wheel_hub.sldprt"
swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warr
swApp.ActivateDoc2("wheel_hub.sldprt", False, errors)

'Create an assembly
swSheetWidth = 0
swSheetHeight = 0
swAssembly = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2018\templates\Assembly.asmdot", 0, swSheetWidth, swShe
AssemblyTitle = swAssembly.GetTitle

'Add the part to the assembly
swInsertedComponent = swAssembly.AddComponent5(fileName, swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSe
swApp.CloseDoc(fileName)
TransformData(0) = 1
TransformData(1) = 0
TransformData(2) = 0
TransformData(3) = 0
TransformData(4) = 1
TransformData(5) = 0
TransformData(6) = 0
TransformData(7) = 0
TransformData(8) = 1
TransformData(9) = 0
TransformData(10) = 0
TransformData(11) = 0
TransformData(12) = 1
TransformData(13) = 0

```

```

TransformData(14) = 0
TransformData(15) = 0
TransformDataObj = TransformData
swMathUtil = swApp.GetMathUtility()
swTransform = swMathUtil.CreateTransform((TransformDataObj))
status = swInsertedComponent.SetTransformAndSolve2(swTransform)

'Open and add another part to the assembly
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\beam with holes.sldprt"
swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_AutoMissingConfig, "", errors, war
swModel = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)
swAssembly = swModel
swInsertedComponent = swAssembly.AddComponent5(fileName, swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSe
swApp.CloseDoc(fileName)
TransformData(0) = 1
TransformData(1) = 0
TransformData(2) = 0
TransformData(3) = 0
TransformData(4) = 1
TransformData(5) = 0
TransformData(6) = 0
TransformData(7) = 0
TransformData(8) = 1
TransformData(9) = -0.179380053887144
TransformData(10) = -0.0894895409187302
TransformData(11) = 0.0744189997389913
TransformData(12) = 1
TransformData(13) = 0
TransformData(14) = 0
TransformData(15) = 0
TransformDataObj = TransformData
swMathUtil = swApp.GetMathUtility()
swTransform = swMathUtil.CreateTransform((TransformDataObj))

'Add a concentric mate
swModelDocExt = swModel.Extension
status = swModelDocExt.SelectByRay(-0.158811046822791, -0.11180606883903, 0.110029416510201, -0.400036026779312, -0.51503807
status = swModelDocExt.SelectByRay(-0.0595820179927387, -0.0282041473137156, 0.0120989536100637, -0.400036026779312, -0.5150
swMate = swAssembly.AddMate5(swMateType_e.swMateCONCENTRIC, swMateAlign_e.swMateAlignALIGNED, False, 0.128546596088087, 0.02
swModel.ClearSelection2(True)
swModel.EditRebuild3()

'Move a component in order to add another mate
status = swModelDocExt.SelectByID2("beam with holes-1@Assem1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
TransformData(0) = 0.999268594246539
TransformData(1) = -0.0382397247969312
TransformData(2) = 0
TransformData(3) = 0.0382397247969312
TransformData(4) = 0.999268594246539
TransformData(5) = 0
TransformData(6) = 0
TransformData(7) = 0
TransformData(8) = 1
TransformData(9) = -0.0817169203602251
TransformData(10) = -0.00405863499514564
TransformData(11) = 0.158954897029145
TransformData(12) = 1
TransformData(13) = 0
TransformData(14) = 0
TransformData(15) = 0
TransformDataObj = TransformData
swMathUtil = swApp.GetMathUtility()
swTransform = swMathUtil.CreateTransform((TransformDataObj))
swSelectionManager = swModel.SelectionManager
swComp = swSelectionManager.GetSelectedObjectsComponent4(1, -1)
status = swComp.SetTransformAndSolve2(swTransform)

'Add a misaligned symmetric concentric mate
status = swModelDocExt.SelectByRay(-0.0362917984178921, -0.0292363280984205, 0.194521666713399, -0.400036026779312, -0.51503
status = swModelDocExt.SelectByRay(0.054967212945769, -0.0278611795938559, 0.0112380964281442, -0.400036026779312, -0.51503
Debug.Print("Misaligned concentric mate:")
swMate = swAssembly.AddConcentricMateWithTolerance(swMateAlign_e.swMateAlignALIGNED, swConcentricAlignmentType_e.swConcentri
If errors = 1 Then
    Debug.Print(" Added")
Else
    Debug.Print(" Not added")
End If

swModel.ClearSelection2(True)

'Edit the misaligned symmetric concentric mate
'and change its position from symmetric to aligned
status = swModelDocExt.SelectByRay(0.0548392523382404, -0.0277425865184, 0.00906375356890976, -0.400036026779312, -0.515038
status = swModelDocExt.SelectByRay(0.00980255664001106, -0.0292099642897483, 0.193646683964346, -0.400036026779312, -0.51503
status = swModelDocExt.SelectByID2("Concentric2", "MATE", 0, 0, 0, True, 0, Nothing, 0)
swAssembly.EditConcentricMate(swMateAlign_e.swMateAlignALIGNED, swConcentricAlignmentType_e.swConcentricAlignThisMate, True,

swModel.ClearSelection2(True)

If errors = 1 Then

```

```

Debug.Print(" Edited")
status = swModelDocExt.SelectByID2("Concentric2", "MATE", 0, 0, 0, True, 0, Nothing, 0)
swFeature = swSelectionManager.GetSelectedObject6(1, -1)
swMate = swFeature.GetSpecificFeature2

'Get the linked concentric mate
swLinkedMate = swMate.GetLinkedMate
'Get current misaligned deviation
Debug.Print(" Current misalignment deviation: " & swLinkedMate.GetCurrentMisalignedDeviation)
'Do not use document property value for misalignment deviation
swMate.SetUseMisalignedDeviationDocumentProperty(False)
Debug.Print(" Use document property? " & swMate.GetUseMisalignedDeviationDocumentProperty)
'Change mate alignment type to symmetric
swLinkedMate.SetConcentricAlignmentType(swConcentricAlignmentType_e.swConcentricAlignSymmetric)
Debug.Print(" Concentric alignment type as defined in swConcentricAlignmentType_e: " & swLinkedMate.GetConcentricAlign
'Set maximum misalignment deviation
swMate.SetMaximumMisalignedDeviation(0.102)
Debug.Print(" Maximum misalignment deviation: " & swMate.GetMaximumMisalignedDeviation)
Else
Debug.Print(" Not edited")
End If

swModel.ClearSelection2(True)
swModel.EditRebuild3()

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Add and Get Custom Properties Example (VBA)

This example shows how to add and get a custom property assigned to a weldment feature.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\weldment_box3.sldprt.
' 2. Select the Weldment feature in the FeatureManager design
'    tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Adds a custom property named Date added to the selected
'    weldment feature.
' 2. To verify:
'    * Examine the Immediate window.
'    * Right-click the Weldment feature in the FeatureManager
'      design and click Properties.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swCustPropMgr As SldWorks.CustomPropertyManager
    Dim nRetVal As Long
    Dim vNameArr As Variant
    Dim vName As Variant
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)
    Set swCustPropMgr = swFeat.CustomPropertyManager

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    ' Add custom property
    nRetVal = swCustPropMgr.Add3("Date added", swCustomInfoType_e.swCustomInfoDate, "17-Apr-2005",
    swCustomPropertyAddOption_e.swCustomPropertyDeleteAndAdd)

    ' Get all custom properties
    vNameArr = swCustPropMgr.GetNames: If IsEmpty(vNameArr) Then Exit Sub
    For Each vName In vNameArr
        Debug.Print " " & vName & " [" & swCustPropMgr.GetType(vName) & "] = " & swCustPropMgr.Get(vName)
    Next vName

End Sub
```

Add and Get Property Extensions Example (C#)

This example shows how to add and get a property extension on a part.

```
//-----
// Preconditions:
// 1. Open a part.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Adds three property extensions to the part.
// 2. Gets the three property extensions added to the part.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            PartDoc swPart = default(PartDoc);
            int retVal = 0;
            string[] objAdded = new string[3];
            string objRetrieved = null;

            swPart = (PartDoc)swApp.ActiveDoc;

            Debug.Print("Property extensions added:");

            objAdded[0] = "12345PID";
            retVal = swPart.AddPropertyExtension(objAdded[0]);
            Debug.Print(" First property extension: " + objAdded[0]);

            objAdded[1] = "6789PID";
            retVal = swPart.AddPropertyExtension(objAdded[1]);
            Debug.Print(" Second property extension: " + objAdded[1]);

            objAdded[2] = "00000PID";
            retVal = swPart.AddPropertyExtension(objAdded[2]);
            Debug.Print(" Third property extension: " + objAdded[2]);

            Debug.Print("Property extensions retrieved:");

            objRetrieved = (string)swPart.GetPropertyExtension(retVal - 3);
            Debug.Print(" First property extension: " + objRetrieved);

            objRetrieved = (string)swPart.GetPropertyExtension(retVal - 2);
            Debug.Print(" Second property extension: " + objRetrieved);

            objRetrieved = (string)swPart.GetPropertyExtension(retVal - 1);
            Debug.Print(" Third property extension: " + objRetrieved);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Add and Get Property Extensions Example (VBA)

This example shows how to add and get a property extension on a part.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Adds three property extensions to the part.  
' 2. Gets the three property extensions added to the part.  
' 3. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swPart As SldWorks.PartDoc  
    Dim retVal As Long  
    Dim objAdded As Variant  
    Dim objAdded2 As Variant  
    Dim objAdded3 As Variant  
    Dim objRetrieved As Variant  
  
    Set swApp = Application.SldWorks  
    Set swPart = swApp.ActiveDoc  
  
    Debug.Print "Property extensions added:"  
  
    objAdded = "12345PID"  
    retVal = swPart.AddPropertyExtension(objAdded)  
    Debug.Print " First property extension: " & objAdded  
  
    objAdded2 = "6789PID"  
    retVal = swPart.AddPropertyExtension(objAdded2)  
    Debug.Print " Second property extension: " & objAdded2  
  
    objAdded3 = "00000PID"  
    retVal = swPart.AddPropertyExtension(objAdded3)  
    Debug.Print " Third property extension: " & objAdded3  
  
    Debug.Print "Property extensions retrieved:"  
  
    objRetrieved = swPart.GetPropertyExtension(retVal - 3)  
    Debug.Print " First property extension: " & objRetrieved  
  
    objRetrieved = swPart.GetPropertyExtension(retVal - 2)  
    Debug.Print " Second property extension: " & objRetrieved  
  
    objRetrieved = swPart.GetPropertyExtension(retVal - 1)  
    Debug.Print " Third property extension: " & objRetrieved  
  
End Sub
```

Add and Get Property Extensions Example (VB.NET)

This example shows how to add and get a property extension on a part.

```
-----
Preconditions:
1. Open a part.
2. Open the Immediate window.
Postconditions:
1. Adds three property extensions to the part.
2. Gets the three property extensions added to the part.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swPart As PartDoc
        Dim retVal As Integer
        Dim objAdded As Object
        Dim objAdded2 As Object
        Dim objAdded3 As Object
        Dim objRetrieved As Object

        swPart = swApp.ActiveDoc

        Debug.Print("Property extensions added:")

        objAdded = "12345PID"
        retVal = swPart.AddPropertyExtension(objAdded)
        Debug.Print(" First property extension: " & objAdded)

        objAdded2 = "6789PID"
        retVal = swPart.AddPropertyExtension(objAdded2)
        Debug.Print(" Second property extension: " & objAdded2)

        objAdded3 = "00000PID"
        retVal = swPart.AddPropertyExtension(objAdded3)
        Debug.Print(" Third property extension: " & objAdded3)

        Debug.Print("Property extensions retrieved:")

        objRetrieved = swPart.GetPropertyExtension(retVal - 3)
        Debug.Print(" First property extension: " & objRetrieved)

        objRetrieved = swPart.GetPropertyExtension(retVal - 2)
        Debug.Print(" Second property extension: " & objRetrieved)

        objRetrieved = swPart.GetPropertyExtension(retVal - 1)
        Debug.Print(" Third property extension: " & objRetrieved)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Add and Offset Dimension Extension Lines Jogs (VBA)

This example shows how to jog the selected dimension's extension lines and to then offset the jogs.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
' Postconditions:
' 1. Selects a linear dimension.
' 2. Jogs the selected dimension's extension lines.
' 3. Offsets the jogs.
' 4. Select the highlighted dimension in the drawing.
' 5. Click anywhere outside the drawing view in the drawing
'    and examine the extension lines of the dimension.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDim As SldWorks.DisplayDimension
Dim swModelView As SldWorks.ModelView
Dim boolstatus As Boolean
Dim index As Integer
Dim jogged As Boolean
Dim offset1 As Double, offset2 As Double, offset1to2 As Double
Dim rect As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelView = swModel.ActiveView
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    boolstatus = swModel.ActivateView("Drawing View1")
    boolstatus = swModelDocExt.SelectByID2("RD9@Drawing View1", "DIMENSION", 0.214177376160146, 0.307386295566502, 0, False, 0, Nothing, 0)
    Set swDisplayDim = swSelMgr.GetSelectedObject6(1, -1)

    ' Add jogs to the dimension's extension lines
    boolstatus = swModelDocExt.JogDimension(True, 0)
    boolstatus = swModelDocExt.JogDimension(True, 1)

    ' Offset the jogs
    index = 0
    boolstatus = swDisplayDim.GetJogParameters(index, jogged, offset1, offset2, offset1to2)
    offset1 = offset1 + offset1 * 0.1
    offset2 = offset2 + offset2 * 0.2
    offset1to2 = offset1to2 + offset1 * 0.2
    boolstatus = swDisplayDim.SetJogParameters(index, jogged, offset1, offset2, offset1to2)

    index = 1
    boolstatus = swDisplayDim.GetJogParameters(index, jogged, offset1, offset2, offset1to2)
    offset1 = offset1 + offset1 * 0.1
    offset2 = offset2 + offset2 * 0.2
    offset1to2 = offset1to2 + offset1 * 0.2
    boolstatus = swDisplayDim.SetJogParameters(index, jogged, offset1, offset2, offset1to2)

    swModelView.GraphicsRedraw ((rect))

End Sub
```

Add and Remove Files from Pack and Go Example (C#)

This example shows how to add SOLIDWORKS Design, render reference, and non-SOLIDWORKS Design files to Pack and Go. This example also shows how to remove a non-SOLIDWORKS Design file from Pack and Go.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly exists.
// 2. Create c:\PackAndGo.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets the names of SOLIDWORKS Design, render reference, and
//    non-SOLIDWORKS Design files for Pack and Go.
// 2. Gets the the name of non-SOLIDWORKS Design file to remove.
// 3. Packs up SOLIDWORKS Design, render reference, and
//    non-SOLIDWORKS Design files and copies them to c:\PackAndGo.
// 4. Examine c:\PackAndGo and the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace AddExternalDocumentsPackAndGo.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            PackAndGo swPackAndGo = default(PackAndGo);
            string openFile = null;
            int namesCount = 0;
            int errors = 0;
            int warnings = 0;
            bool status = false;
            int i = 0;
            object[] renderReferences = null;
            string myPath = null;
            object statuses = null;

            // Open assembly document
            openFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2025\\samples\\tutorial\\EDraw\\claw\\claw-
mechanism.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(openFile, (int)swDocumentTypes_e.swDocASSEMBLY,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Get Pack and Go object
            Debug.Print("Pack and Go");
            swPackAndGo = (PackAndGo)swModelDocExt.GetPackAndGo();

            // Get number of documents in assembly
            namesCount = swPackAndGo.GetDocumentNamesCount();

            // Get current paths and filenames of the assembly's documents
            object fileNames;
            object[] pgFileNames = new object[namesCount - 1];
            status = swPackAndGo.GetDocumentNames(out fileNames);
            pgFileNames = (object[])fileNames;

            Debug.Print("");
            Debug.Print(" Add SOLIDWORKS Design files' paths and filenames: ");
            if ((pgFileNames != null))
            {
                for (i = 0; i <= pgFileNames.GetUpperBound(0); i++)
                {
                    Debug.Print(" The path and filename is: " + pgFileNames[i]);
                }
            }

            // Set document paths and names for Pack and Go
            status = swPackAndGo.SetDocumentSaveToNames(pgFileNames);

            // Get the render stock references in this assembly
            // and print them to the Immediate window
            Debug.Print(" ");
            renderReferences = (object[])swModelDocExt.GetRenderStockReferences();
            Debug.Print(" Add render references:");
            for (i = 0; i <= renderReferences.GetUpperBound(0); i++)
            {
                Debug.Print(" The path and filename is: " + renderReferences[i]);
            }

            // Add render stock files to Pack and Go
            status = swPackAndGo.AddExternalDocuments(renderReferences);
        }
    }
}
```

```

        // Add other non-SOLIDWORKS Design files to Pack and Go
        object[] otherFiles = new object[2];
        string otherFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2025\\samples\\tutorial\\edraw\\claw\\claw-
mechanism.easm";
        otherFiles[0] = (object)otherFile;
        otherFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2025\\samples\\tutorial\\edraw\\claw\\claw-
mechanism.emodel_debugonly.xml";
        otherFiles[1] = (object)otherFile;

        Debug.Print(" ");
        Debug.Print(" Add non-SOLIDWORKS Design files:");
        for (i = 0; i <= otherFiles.GetUpperBound(0); i++)
        {
            Debug.Print(" The path and filename is: " + otherFiles[i]);
        }

        // Add non-SOLIDWORKS Design file to Pack and Go
        status = swPackAndGo.AddExternalDocuments(ObjectArrayToBStrWrapperArray(otherFiles));

        // Remove one of the non-SOLIDWORKS Design files from Pack and Go
        object[] delOtherFiles = new object[1];
        delOtherFiles[0] = (object)otherFiles[0];

        Debug.Print(" ");
        Debug.Print(" Remove non-SOLIDWORKS Design file:");
        Debug.Print(" The path and filename is: " + delOtherFiles[0]);

        status = swPackAndGo.RemoveExternalDocuments(ObjectArrayToBStrWrapperArray(delOtherFiles));

        // Override path where to save documents
        myPath = "c:\\PackAndGo\\";
        status = swPackAndGo.SetSaveToName2(true, myPath);

        // Pack and Go both SOLIDWORKS and non-SOLIDWORKS files
        statuses = swModelDocExt.SavePackAndGo(swPackAndGo);
    }

    public BStrWrapper[] ObjectArrayToBStrWrapperArray(object[] SwObjects)
    {
        int arraySize;
        arraySize = SwObjects.GetUpperBound(0);
        BStrWrapper[] dispwrap = new BStrWrapper[arraySize + 1];
        int arrayIndex;

        for (arrayIndex = 0; arrayIndex < arraySize + 1; arrayIndex++)
        {
            dispwrap[arrayIndex] = new BStrWrapper((string)(SwObjects[arrayIndex]));
        }

        return dispwrap;
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}

```

Add and Remove Files from Pack and Go Example (VBA)

This example shows how to add SOLIDWORKS Design, render reference, and non-SOLIDWORKS Design files to Pack and Go. This example also shows how to remove a non-SOLIDWORKS Design file from Pack and Go.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Create c:\PackAndGo.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the names of SOLIDWORKS Design, render reference, and
'    non-SOLIDWORKS Design files for Pack and Go.
' 2. Gets the the name of non-SOLIDWORKS Design file to remove.
' 3. Packs up SOLIDWORKS Design, render reference, and
'    non-SOLIDWORKS Design files and copies them to c:\PackAndGo.
' 4. Examine c:\PackAndGo and the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swPackAndGo As SldWorks.PackAndGo
Dim openFile As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean
Dim pgFileNames As Variant
Dim i As Long
Dim renderReferences As Variant
Dim otherFiles(1) As String
Dim delOtherFiles(0) As String
Dim myPath As String
Dim statuses As Variant

Sub main()

Set swApp = Application.SldWorks

' Open assembly document
openFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\EDraw\claw\claw-mechanism.sldasm"
Set swModel = swApp.OpenDoc6(openFile, swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension

' Get Pack and Go object
Debug.Print "Pack and Go"
Set swPackAndGo = swModelDocExt.GetPackAndGo

' Get current paths and filenames of the assembly's documents
status = swPackAndGo.GetDocumentNames(pgFileNames)
Debug.Print ""
Debug.Print " Add SOLIDWORKS files' paths and filenames: "
If (Not (IsEmpty(pgFileNames))) Then
    For i = 0 To UBound(pgFileNames)
        Debug.Print " The path and filename is: " & pgFileNames(i)
    Next i
End If

' Set document paths and names for Pack and Go
status = swPackAndGo.SetDocumentSaveToNames(pgFileNames)

' Get the render stock references in this assembly
' and print them to the Immediate window
Debug.Print ""
renderReferences = swModelDocExt.GetRenderStockReferences
Debug.Print " Add render references:"
For i = 0 To UBound(renderReferences)
    Debug.Print " The path and filename is: " & renderReferences(i)
Next i

' Add render stock files to Pack and Go
status = swPackAndGo.AddExternalDocuments(renderReferences)

' Add other non-SOLIDWORKS Design files to Pack and Go
otherFiles(0) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\edraw\claw\claw-mechanism.easm"
otherFiles(1) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\edraw\claw\claw-mechanism.emodel_debugonly.xml"
Debug.Print ""
Debug.Print " Add non-SOLIDWORKS Design files:"
For i = 0 To UBound(otherFiles)
    Debug.Print " The path and filename is: " & otherFiles(i)
Next i
status = swPackAndGo.AddExternalDocuments(otherFiles)

' Remove one of the non-SOLIDWORKS Design files from Pack and Go
Debug.Print ""
Debug.Print " Remove non-SOLIDWORKS Design file:"
delOtherFiles(0) = otherFiles(0)
Debug.Print " The path and filename is: " & delOtherFiles(0)
status = swPackAndGo.RemoveExternalDocuments(delOtherFiles)

' Override path where to save documents
myPath = "c:\PackAndGo\"
status = swPackAndGo.SetSaveToName2(True, myPath)

' Pack and Go both SOLIDWORKS and non-SOLIDWORKS files
statuses = swModelDocExt.SavePackAndGo(swPackAndGo)
```

End Sub

Add and Remove Files from Pack and Go Example (VB.NET)

This example shows how to add SOLIDWORKS Design, render reference, and non-SOLIDWORKS Design files to Pack and Go. This example also shows how to remove a non-SOLIDWORKS Design file from Pack and Go.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Create c:\PackAndGo.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the names of SOLIDWORKS Design, render reference, and
'    non-SOLIDWORKS Design files for Pack and Go.
' 2. Gets the the name of non-SOLIDWORKS Design file to remove.
' 3. Packs up SOLIDWORKS Design, render reference, and
'    non-SOLIDWORKS Design files and copies them to c:\PackAndGo.
' 4. Examine c:\PackAndGo and the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swPackAndGo As PackAndGo
        Dim openFile As String
        Dim errors As Long
        Dim warnings As Long
        Dim status As Boolean
        Dim pgFileNames As Object
        Dim i As Long
        Dim renderReferences As Object
        Dim otherFiles(1) As String
        Dim delOtherFiles(0) As String
        Dim myPath As String
        Dim statuses As Object

        ' Open assembly document
        openFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\EDraw\claw\claw-mechanism.sldasm"
        swModel = swApp.OpenDoc6(openFile, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        ' Get Pack and Go object
        Debug.Print("Pack and Go")
        swPackAndGo = swModelDocExt.GetPackAndGo

        ' Get current paths and filenames of the assembly's documents
        status = swPackAndGo.GetDocumentNames(pgFileNames)
        Debug.Print("")
        Debug.Print("  Add SOLIDWORKS files' paths and filenames: ")
        If Not IsNothing(pgFileNames) Then
            For i = 0 To UBound(pgFileNames)
                Debug.Print("    The path and filename is: " & pgFileNames(i))
            Next i
        End If

        ' Set document paths and names for Pack and Go
        status = swPackAndGo.SetDocumentSaveToNames(pgFileNames)

        ' Get the render stock references in this assembly
        ' and print them to the Immediate window
        Debug.Print(" ")
        renderReferences = swModelDocExt.GetRenderStockReferences
        Debug.Print("  Add render references:")
        For i = 0 To UBound(renderReferences)
            Debug.Print("    The path and filename is: " & renderReferences(i))
        Next i

        ' Add render stock files to Pack and Go
        status = swPackAndGo.AddExternalDocuments(renderReferences)

        ' Add other non-SOLIDWORKS Design files to Pack and Go
        otherFiles(0) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\edraw\claw\claw-mechanism.easm"
        otherFiles(1) = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\edraw\claw\claw-mechanism.emodel_debugonly.xml"
        Debug.Print(" ")

        Debug.Print("  Add non-SOLIDWORKS Design files:")

        For i = 0 To UBound(otherFiles)
            Debug.Print("    The path and filename is: " & otherFiles(i))
        Next i

        status = swPackAndGo.AddExternalDocuments(otherFiles)

        ' Remove one of the non-SOLIDWORKS Design files from Pack and Go
        Debug.Print(" ")
        Debug.Print("  Remove non-SOLIDWORKS Design file:")
        delOtherFiles(0) = otherFiles(0)
        Debug.Print("    The path and filename is: " & delOtherFiles(0))
        status = swPackAndGo.RemoveExternalDocuments(delOtherFiles)
    End Sub
End Class

```

```
' Override path where to save documents
myPath = "c:\PackAndGo\"
status = swPackAndGo.SetSaveToName2(True, myPath)

' Pack and Go both SOLIDWORKS Design and non-SOLIDWORKS Design files
statuses = swModelDocExt.SavePackAndGo(swPackAndGo)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

Public swApp As SldWorks

End Class
```

Add and Remove Items to File Save As and Open Menus Example (C#)

This example shows how to add items to the **File > Save As** and **File > Open** file type menus.

```
//-----
// Preconditions:
// 1. In Microsoft Visual Studio 2010, or later, create an SOLIDWORKS add-in
//    project using Visual C# > SwCSharpAddin.
// 2. Name the project SwCSharpAddin1.
// 3. Copy and paste this code into SwAddin.cs of your C# project.
// 4. Click Project > SwCSharpAddin1 Properties > Debug.
// 5. Select Start external program and type the path to sldworks.exe. Include
//    sldworks.exe in the path.
// 6. Press F5 to start debugging SOLIDWORKS Design with this add-in.
//
// Postconditions:
// 1. Open a part.
// 2. Click File > Save As.
// 3. Select XXX file (*.xxx) in Save as type.
// 4. Click Save.
// 5. Displays the path and file name to which to save the part.
// 6. Click OK to close the message box.
//-----
// SwAddin.cs:
using System;
using System.Runtime.InteropServices;
using System.Collections;
using System.Reflection;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swpublished;
using SolidWorks.Interop.swconst;
using SolidWorksTools;
using SolidWorksTools.File;

using System.Windows.Forms;

namespace SwCSharpAddin1
{
    /// <summary>
    /// Create third-party pop-up menus.
    /// </summary>
    [Guid("144d6667-43ec-4733-ae26-867c9aa63fe6"), ComVisible(true)]
    [SwAddin(
        Description = "SwCSharpAddin1 description",
        Title = "SwCSharpAddin1",
        LoadAtStartup = true
    )]
    public class SwAddin : ISwAddin
    {
        #region Local Variables
        ISldWorks iSwApp;
        ICommandManager iCmdMgr;
        int addinID;
        int registerID;
        bool resultCode;
        IModelDoc2 model;
        IFrame frame;

        #region Event Handler Variables
        Hashtable openDocs;
        SolidWorks.Interop.sldworks.SldWorks SwEventPtr;
        #endregion

        #region Property Manager Variables
        UserPMPage ppage;
        #endregion

        // Public Properties
        public ISldWorks SwApp
        {
            get { return iSwApp; }
        }
        public ICommandManager CmdMgr
        {
            get { return iCmdMgr; }
        }

        public Hashtable OpenDocs
        {
            get { return openDocs; }
        }

        #endregion

        #region SOLIDWORKS Registration
```

```

[ComRegisterFunctionAttribute]
public static void RegisterFunction(Type t)
{
    #region Get Custom Attribute: SwAddinAttribute
    SwAddinAttribute SwAttr = null;
    Type type = typeof(SwAddin);
    foreach (System.Attribute attr in type.GetCustomAttributes(false))
    {
        if (attr is SwAddinAttribute)
        {
            SwAttr = attr as SwAddinAttribute;
            break;
        }
    }
    #endregion

    Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
    Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

    string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "}";
    Microsoft.Win32.RegistryKey addinkey = hklm.CreateSubKey(keyname);
    addinkey.SetValue(null, 0);

    addinkey.SetValue("Description", SwAttr.Description);
    addinkey.SetValue("Title", SwAttr.Title);

    keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "}";
    addinkey = hkcu.CreateSubKey(keyname);
    addinkey.SetValue(null, Convert.ToInt32(SwAttr.LoadAtStartup), Microsoft.Win32.RegistryValueKind.DWord);
}

[ComUnregisterFunctionAttribute]
public static void UnregisterFunction(Type t)
{
    Microsoft.Win32.RegistryKey hklm = Microsoft.Win32.Registry.LocalMachine;
    Microsoft.Win32.RegistryKey hkcu = Microsoft.Win32.Registry.CurrentUser;

    string keyname = "SOFTWARE\\SOLIDWORKS\\Addins\\" + t.GUID.ToString() + "}";
    hklm.DeleteSubKey(keyname);

    keyname = "Software\\SOLIDWORKS\\AddInsStartup\\" + t.GUID.ToString() + "}";
    hkcu.DeleteSubKey(keyname);
}

#endregion

#region ISwAddin Implementation
public SwAddin()
{
}

public bool ConnectToSW(object ThisSW, int cookie)
{
    iSwApp = (ISldWorks)ThisSW;
    addinID = cookie;

    //Set up callbacks
    iSwApp.SetAddinCallbackInfo(0, this, addinID);

    //Set up File > Save As > Save as type item and File > Open > File name type item
    bRet = iSwApp.AddFileSaveAsItem2(addinID, "XXX_FileSave", "XXX file (*.xxx)", "xxx", (int)swDocumentTypes_e.swDocPART);
    bRet = iSwApp.AddFileOpenItem3(addinID, "XXX_FileOpen", "XXX file (*.xxx)", "xxx", "", "");

    #region Setup the Command Manager
    iCmdMgr = iSwApp.GetCommandManager(cookie);
    AddCommandMgr();
    #endregion

    #region Setup the Event Handlers
    SwEventPtr = (SolidWorks.Interop.sldworks.SldWorks)iSwApp;
    openDocs = new Hashtable();
    AttachEventHandlers();
    #endregion

    #region Setup Sample Property Manager
    AddPMP();
    #endregion

    return true;
}

public bool DisconnectFromSW()
{
    bRet = iSwApp.RemoveFileSaveAsItem2(addinID, "XXX_FileSave", "XXX file (*.xxx)", "xxx", (int)swDocumentTypes_e.swDocPART);
    bRet = iSwApp.RemoveFileOpenItem2(addinID, "XXX_FileOpen", "XXX file (*.xxx)", "xxx");

    RemoveCommandMgr();
    RemovePMP();
    DetachEventHandlers();

    iSwApp = null;
    //The addin _must_ call GC.Collect() here in order to retrieve all managed code pointers
}

```

```

        GC.Collect();
        return true;
    }

    public void XXX_FileSave(string sFileName)
    {
        // S_OK          = Saved successfully
        // S_FALSE       = Unsuccessful
        // !(SUCCEEDED) = Insuccessful

        int fileOp = 1;
        ParseFilename(sFileName, ref fileOp);
    }

    public void XXX_FileOpen(string sFileName)
    {
        // S_OK          = Loaded successfully
        // S_FALSE       = Unsuccessful
        // !(SUCCEEDED) = Unsuccessful
        int fileOp = 0;
        ParseFilename(sFileName, ref fileOp);
    }

    public void ParseFilename(string sFileName, ref int fileOp)
    {
        string strExtension = null;
        int lPos = 0;
        int lNumExtensionOccurrences = 0;
        int lStart = 0;
        string strSearchString = null;
        int lNumNonRealExtensions = 0;
        string strExtensionWithPeriod = null;

        // Set the extension for which to look
        strExtension = "xxx";
        strExtensionWithPeriod = "." + strExtension;

        // Strip the trailing 'w' or 'r' and any leading and trailing white space
        sFileName = (sFileName.Substring(0, sFileName.Length - 1)).Trim(' ');

        // Strip extension from the back
        sFileName = (sFileName.Substring(0, sFileName.Length - strExtension.Length)).Trim(' ');

        // Change to lowercase to make search case-insensitive
        strSearchString = sFileName.ToLower();

        lNumExtensionOccurrences = 0;

        lStart = 1;

        do
        {
            lPos = strSearchString.IndexOf(strExtensionWithPeriod.ToLower(), lStart - 1) + 1;
            if (lPos > 0)
            {
                lNumExtensionOccurrences = lNumExtensionOccurrences + 1;

                // Move start point of search
                lStart = (int)(lPos + strExtensionWithPeriod.Length);
            }
        } while (lPos > 0);

        // There is 1 real extension and n*2 non-real extension
        lNumNonRealExtensions = (lNumExtensionOccurrences / 2);

        // Start searching from the end to locate the real extension
        // Skip the number of non-real extensions, before reaching the real extension

        // Change to lowercase to make search case-insensitive
        strSearchString = sFileName.ToLower();

        lPos = strSearchString.LastIndexOf(strExtensionWithPeriod.ToLower(), lStart);

        sFileName = sFileName.Substring(0, lPos) + strExtensionWithPeriod;

        if (fileOp == 1)
        {
            MessageBox.Show("XXX_FileSave = " + sFileName);
        }
        else
        {
            MessageBox.Show("XXX_FileOpen = " + sFileName);
        }
    }
}

#endregion

#region UI Methods

```

```

public void AddCommandMgr()
{
    ICommandGroup cmdGroup;
    BitmapHandler iBmp = new BitmapHandler();
    Assembly thisAssembly;
    int cmdIndex0, cmdIndex1;
    string Title = "C# Add-in", ToolTip = "C# Add-in";

    int[] docTypes = new int[] {(int)swDocumentTypes_e.swDocASSEMBLY,
                                (int)swDocumentTypes_e.swDocDRAWING,
                                (int)swDocumentTypes_e.swDocPART};

    thisAssembly = System.Reflection.Assembly.GetAssembly(this.GetType());

    cmdGroup = iCmdMgr.CreateCommandGroup(1, Title, ToolTip, "", -1);
    cmdGroup.LargeIconList = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.ToolbarLarge.bmp", thisAssembly);
    cmdGroup.SmallIconList = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.ToolbarSmall.bmp", thisAssembly);
    cmdGroup.LargeMainIcon = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.MainIconLarge.bmp", thisAssembly);
    cmdGroup.SmallMainIcon = iBmp.CreateFileFromResourceBitmap("SwCSharpAddin1.MainIconSmall.bmp", thisAssembly);

    cmdIndex0 = cmdGroup.AddCommandItem("CreateCube", -1, "Create a cube", "Create cube", 0, "CreateCube", "", 0);
    cmdIndex1 = cmdGroup.AddCommandItem("Show PMP", -1, "Display sample property manager", "Show PMP", 2, "ShowPMP",
"EnablePMP", 2);

    cmdGroup.HasToolbar = true;
    cmdGroup.HasMenu = true;
    cmdGroup.Activate();

    bool bResult;

    foreach (int type in docTypes)
    {
        ICommandTab cmdTab;

        cmdTab = iCmdMgr.GetCommandTab(type, Title);

        if (cmdTab == null)
        {
            cmdTab = (ICommandTab)iCmdMgr.AddCommandTab(type, Title);

            CommandTabBox cmdBox = cmdTab.AddCommandTabBox();

            int[] cmdIDs = new int[3];
            int[] TextType = new int[3];

            cmdIDs[0] = cmdGroup.get_CommandID(cmdIndex0);
            System.Diagnostics.Debug.Print(cmdGroup.get_CommandID(cmdIndex0).ToString());
            TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

            cmdIDs[1] = cmdGroup.get_CommandID(cmdIndex1);
            System.Diagnostics.Debug.Print(cmdGroup.get_CommandID(cmdIndex1).ToString());
            TextType[1] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal;

            cmdIDs[2] = cmdGroup.ToolbarId;
            System.Diagnostics.Debug.Print(cmdIDs[2].ToString());
            TextType[2] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextHorizontal |
(int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

            bResult = cmdBox.AddCommands(cmdIDs, TextType);

            CommandTabBox cmdBox1 = cmdTab.AddCommandTabBox();
            cmdIDs = new int[1];
            TextType = new int[1];

            cmdIDs[0] = cmdGroup.ToolbarId;
            TextType[0] = (int)swCommandTabButtonTextDisplay_e.swCommandTabButton_TextBelow |
(int)swCommandTabButtonFlyoutStyle_e.swCommandTabButton_ActionFlyout;

            bResult = cmdBox1.AddCommands(cmdIDs, TextType);

            cmdTab.AddSeparator(cmdBox1, cmdGroup.ToolbarId);

        }
    }

    // create a third-party icon in the context-sensitive menus of faces in parts
    frame = (IFrame)SwApp.Frame();
    resultCode = frame.AddMenuPopupIcon2((int)swDocumentTypes_e.swDocPART, (int)swSelectType_e.swSelFACES,
"contextsensitive", addinID, "CSCallbackFunction", "CSEnable", "", cmdGroup.SmallMainIcon);

    // create and register the third party menu
    registerID = SwApp.RegisterThirdPartyPopupMenu();

    // add a menu break at the top of the menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Menu Break", addinID,
"", "", "", "", (int)swMenuItemType_e.swMenuItemType_Break
);
    // add a couple of items to the menu
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Test1", addinID,
"TestCallback", "EnableTest", "", "Test1", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
);
    resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Test4", addinID,

```

```

"TestCallback", "EnableTest", "", "Test4", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
);
// add a separator bar to the menu
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "", addinID, "", "", "",
"", "", (int)swMenuItemType_e.swMenuItemType_Separator);
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "Test5", addinID,
"TestCallback", "EnableTest", "", "Test5", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
);

// add an icon to the menu bar
resultCode = SwApp.AddItemToThirdPartyPopupMenu2(registerID, (int)swDocumentTypes_e.swDocPART, "", addinID,
"TestCallback", "EnableTest", "", "NoOp", cmdGroup.SmallMainIcon, (int)swMenuItemType_e.swMenuItemType_Default
);

thisAssembly = null;
iBmp.Dispose();
}

public void RemoveCommandMgr()
{
    iCmdMgr.RemoveCommandGroup(1);
}

public Boolean AddPMP()
{
    ppage = new UserPMPPage(this);
    return true;
}

public Boolean RemovePMP()
{
    ppage = null;
    return true;
}

#endregion

#region UI Callbacks
public void CreateCube()
{
    //make sure we have a part open
    string partTemplate = iSwApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swDefaultTemplatePart);
    IModelDoc2 modDoc = (IModelDoc2)iSwApp.NewDocument(partTemplate, (int)swDwgPaperSizes_e.swDwgPaperA2size, 0.0, 0.0);

    modDoc.InsertSketch2(true);
    modDoc.SketchRectangle(0, 0, 0, .1, .1, .1, false);
    //Extrude the sketch
    IFeatureManager featMan = modDoc.FeatureManager;
    featMan.FeatureExtrusion(true,
        false, false,
        (int)swEndConditions_e.swEndCondBlind, (int)swEndConditions_e.swEndCondBlind,
        0.1, 0.0,
        false, false,
        false, false,
        0.0, 0.0,
        false, false,
        false, false,
        true,
        false, false);
}

public void ShowPMP()
{
    if (ppage != null)
        ppage.Show();
}

public int EnablePMP()
{
    if (iSwApp.ActiveDoc != null)
        return 1;
    else
        return 0;
}

public void CSCallbackFunction()
{
    resultCode = SwApp.ShowThirdPartyPopupMenu(registerID, 500, 500);
}

public int CSEnable()
{
    if (iSwApp.ActiveDoc == null)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}

public void TestCallback()
{
    System.Diagnostics.Debug.Print("Test callback");
}

```

```

    }

    public int EnableTest()
    {
        if (iSwApp.ActiveDoc == null)
        {
            return 0;
        }
        else
        {
            return 1;
        }
    }
}

#endregion

#region Event Methods
public bool AttachEventHandlers()
{
    AttachSwEvents();
    //Listen for events on all currently open docs
    AttachEventsToAllDocuments();
    return true;
}

private bool AttachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify += new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 += new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 += new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify += new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify += new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        DSldWorksEvents_InterfaceBrightnessThemeChangeNotify += new
        DSldWorksEvents_InterfaceBrightnessThemeChangeNotifyEventHandler(InterfaceBrightnessThemeChangeNotify);

        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

private bool DetachSwEvents()
{
    try
    {
        SwEventPtr.ActiveDocChangeNotify -= new DSldWorksEvents_ActiveDocChangeNotifyEventHandler(OnDocChange);
        SwEventPtr.DocumentLoadNotify2 -= new DSldWorksEvents_DocumentLoadNotify2EventHandler(OnDocLoad);
        SwEventPtr.FileNewNotify2 -= new DSldWorksEvents_FileNewNotify2EventHandler(OnFileNew);
        SwEventPtr.ActiveModelDocChangeNotify -= new DSldWorksEvents_ActiveModelDocChangeNotifyEventHandler(OnModelChange);
        SwEventPtr.FileOpenPostNotify -= new DSldWorksEvents_FileOpenPostNotifyEventHandler(FileOpenPostNotify);
        DSldWorksEvents_InterfaceBrightnessThemeChangeNotify -= new
        DSldWorksEvents_InterfaceBrightnessThemeChangeNotifyEventHandler(InterfaceBrightnessThemeChangeNotify);

        return true;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        return false;
    }
}

public void AttachEventsToAllDocuments()
{
    ModelDoc2 modDoc = (ModelDoc2)iSwApp.GetFirstDocument();
    while (modDoc != null)
    {
        if (!openDocs.Contains(modDoc))
        {
            AttachModelDocEventHandler(modDoc);
        }
        modDoc = (ModelDoc2)modDoc.GetNext();
    }
}

public bool AttachModelDocEventHandler(ModelDoc2 modDoc)
{
    if (modDoc == null)
        return false;

    DocumentEventHandler docHandler = null;

    if (!openDocs.Contains(modDoc))
    {
        switch (modDoc.GetType())
        {
            case (int)swDocumentTypes_e.swDocPART:
                {

```

```

        docHandler = new PartEventHandler(modDoc, this);
        break;
    }
    case (int)swDocumentTypes_e.swDocASSEMBLY:
    {
        docHandler = new AssemblyEventHandler(modDoc, this);
        break;
    }
    case (int)swDocumentTypes_e.swDocDRAWING:
    {
        docHandler = new DrawingEventHandler(modDoc, this);
        break;
    }
    default:
    {
        return false; //Unsupported document type
    }
}
docHandler.AttachEventHandlers();
openDocs.Add(modDoc, docHandler);
}
return true;
}

public bool DetachModelEventHandler(ModelDoc2 modDoc)
{
    DocumentEventHandler docHandler;
    docHandler = (DocumentEventHandler)openDocs[modDoc];
    openDocs.Remove(modDoc);
    modDoc = null;
    docHandler = null;
    return true;
}

public bool DetachEventHandlers()
{
    DetachSwEvents();

    //Close events on all currently open docs
    DocumentEventHandler docHandler;
    int numKeys = openDocs.Count;
    object[] keys = new Object[numKeys];

    //Remove all document event handlers
    openDocs.Keys.CopyTo(keys, 0);
    foreach (ModelDoc2 key in keys)
    {
        docHandler = (DocumentEventHandler)openDocs[key];
        docHandler.DetachEventHandlers(); //This also removes the pair from the hash
        docHandler = null;
    }
    return true;
}
#endregion

#region Event Handlers
//Events
public int OnDocChange()
{
    return 0;
}

public int OnDocLoad(string docTitle, string docPath)
{
    return 0;
}

int FileOpenPostNotify(string FileName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnFileNew(object newDoc, int docType, string templateName)
{
    AttachEventsToAllDocuments();
    return 0;
}

public int OnModelChange()
{
    return 0;
}

public int InterfaceBrightnessThemeChangeNotify(int theme, ref object colors)
{
    MessageBox.Show("Interface brightness theme changed to (0 = light, 1 = medium, 2 = dark): " + theme);
    int[] colorsArray = null;
    colorsArray = (int[])colors;
    MessageBox.Show("FeatureManager design tree color: " + colorsArray[0]);

    return 0;
}

```

```
} #endregion  
}
```